



UNIVERSIDADE FEDERAL DE MINAS GERAIS
PROGRAMA DE PÓS-GRADUAÇÃO EM
ENGENHARIA DE ESTRUTURAS
EES847 - ANÁLISE NÃO-LINEAR GEOMÉTRICO
DE ESTRUTURAS RETICULADAS



Pedro Henrique Reis da Silva Lima

TRABALHOS PRÁTICOS DA DISCIPLINA DE ANÁLISE
NÃO-LINEAR GEOMÉTRICO DE ESTRUTURAS RETICULADAS

LISTA DE ILUSTRAÇÕES

Figura 1	–	Sistema estrutural do Trabalho Prático 01.	4
Figura 2	–	TP01 – Curvas do carregamento vertical P	5
Figura 3	–	Sistema estrutural do Trabalho Prático 02.	6
Figura 4	–	TP02 – Curvas do carregamento vertical P	7
Figura 5	–	Sistema estrutural do Trabalho Prático 03.	8
Figura 6	–	TP03 – Curvas do carregamento vertical P	9
Figura 7	–	Sistema estrutural do Trabalho Prático 04.	10
Figura 8	–	TP04 – Curvas do momento fletor M	12
Figura 9	–	TP04 – Linha neutra $v(x)$ para $\theta = \pi$ e para $\theta = 2\pi$	13
Figura 10	–	Sistema estrutural do Trabalho Prático 05.	14
Figura 11	–	TP05 – Curvas da treliça de von Mises.	16
Figura 12	–	TP06 – Medidas clássicas de deformação em função do estiramento. . .	18
Figura 13	–	TP06 – Conjugados de tensão normalizados em função do estiramento. .	18
Figura 14	–	Sistema estrutural do Trabalho Prático 08 – Vista superior	22
Figura 15	–	Sistema estrutural do Trabalho Prático 08 – Vista lateral	23
Figura 16	–	TP08 – Curvas do carregamento vertical P	27
Figura 17	–	Sistema estrutural do Trabalho Prático 09.	28
Figura 18	–	Treliça de dois membros.	33
Figura 19	–	Resultados gerais do Exemplo 1 para o primeiro caso de carregamento. .	34
Figura 20	–	Resultados gerais do Exemplo 1 para o segundo caso de carregamento. .	35
Figura 21	–	Treliça de dois membros.	35
Figura 22	–	Resultados gerais do Exemplo 2.	36

LISTA DE TABELAS

Tabela 1	–	Medidas de deformação clássicas e pares conjugados de tensão em função do estiramento.	17
----------	---	--	----

SUMÁRIO

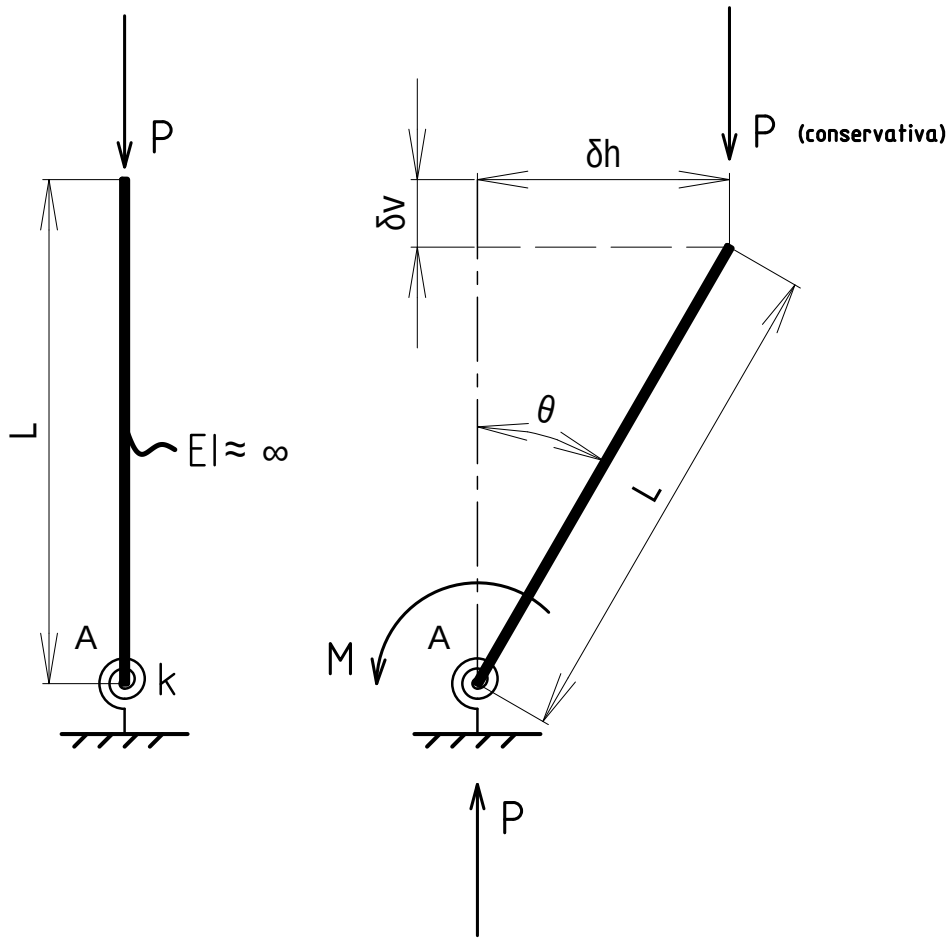
1	Trabalho Prático 01	4
2	Trabalho Prático 02	6
3	Trabalho Prático 03	8
4	Trabalho Prático 04	10
5	Trabalho Prático 05	14
6	Trabalho Prático 06	17
7	Trabalho Prático 07	19
8	Trabalho Prático 08	22
9	Trabalho Prático 09	28
10	Trabalho Prático 10	31
APÊNDICE A - ROTINAS NUMÉRICAS IMPLEMENTADAS PARA SOLUÇÃO DOS TRABALHOS PRÁTICOS		37
A.1 - Início do documento, classes e métodos auxiliares e main		37
A.2 - Trabalho prático 01		41
A.3 - Trabalho prático 02		41
A.4 - Trabalho prático 03		41
A.5 - Trabalho prático 04		42
A.6 - Trabalho prático 05		43
A.7 - Trabalho prático 06		43
A.8 - Trabalho prático 08		44
A.9 - Trabalho prático 09		45

1 TRABALHO PRÁTICO 01

Fazer o gráfico completo da barra rígida conectada à base com a seguinte configuração:

$$\begin{cases} k_{\text{mola}} = 1000 \text{ kN} \cdot \text{cm/rad} \\ L = 100 \text{ cm} \\ \Delta\theta = \frac{\pi}{90} \text{ rad} \end{cases}$$

Figura 1: Sistema estrutural do Trabalho Prático 01.



A relação constitutiva da mola é assumida como linear, de modo que o momento gerado se relaciona com o ângulo de giro θ da barra a partir da Equação 1.1

$$M = k_{\text{mola}} \cdot \theta \quad (1.1)$$

Além disso, há compatibilidade cinemática entre os deslocamentos vertical (δ_v) e horizontal (δ_h) a partir de relações trigonométricas no equilíbrio da configuração deformada:

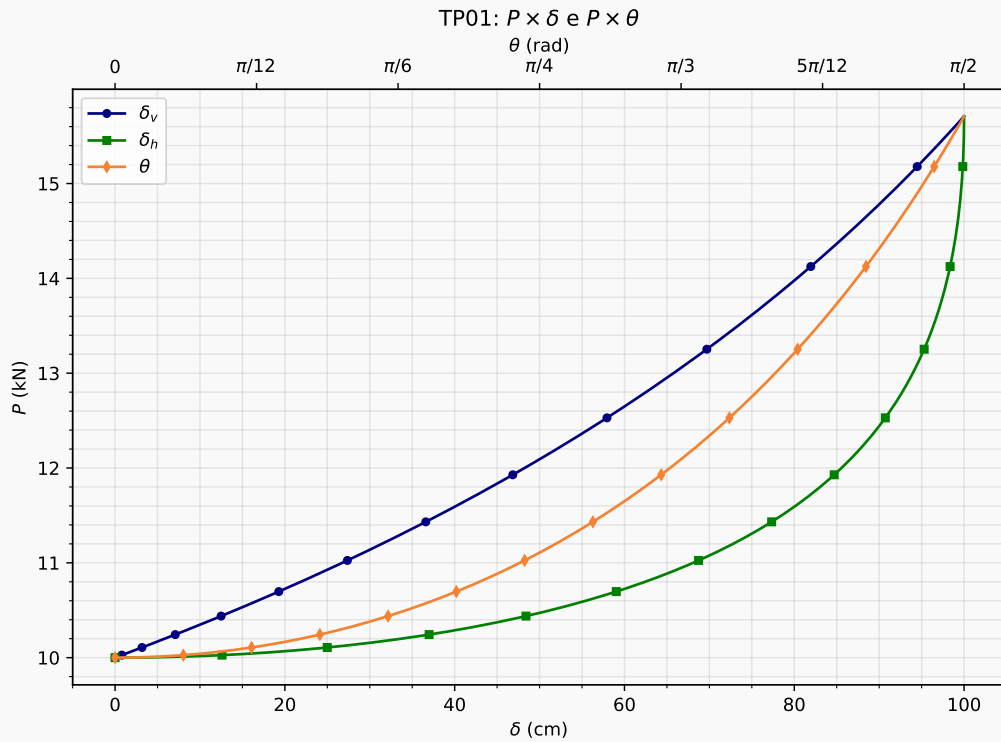
$$\begin{cases} \delta_v = L(1 - \cos \theta) \\ \delta_h = L \sin \theta \end{cases} \quad (1.2)$$

Sendo assim, o sistema estrutural da barra rígida acoplada à uma mola de torção na base possui como solução condição de equilíbrio na configuração deformada a expressão dada na Equação 1.3, assumindo não-linearidade geométrica.

$$P = \frac{k_{\text{mola}}}{L} \cdot \frac{\theta}{\sin \theta} \quad (1.3)$$

Foram geradas as curvas $P \times \theta$, $P \times \delta_v$ e $P \times \delta_h$ em rotina numérica implementada no JavaScript, para os dados fornecidos no enunciado, as quais são mostradas na Figura 2. As linhas de código que geraram estes gráficos estão disponíveis no Apêndice A.2.

Figura 2: TP01 – Curvas do carregamento vertical P .



As curvas evidenciam o comportamento NLG do sistema reticulado, no qual, em um ponto específico da curva, há bifurcação do comportamento da estrutura quanto à estabilidade: a partir deste ponto, denominado "ponto crítico", associado a um valor de carga $P = P_{cr}$, a estrutura assume um comportamento estável de ganho de rigidez em proporção aos deslocamento vertical. O valor de P_{cr} , a saber, é obtido pelo limite para quando o valor do ângulo de giro θ tende a zero:

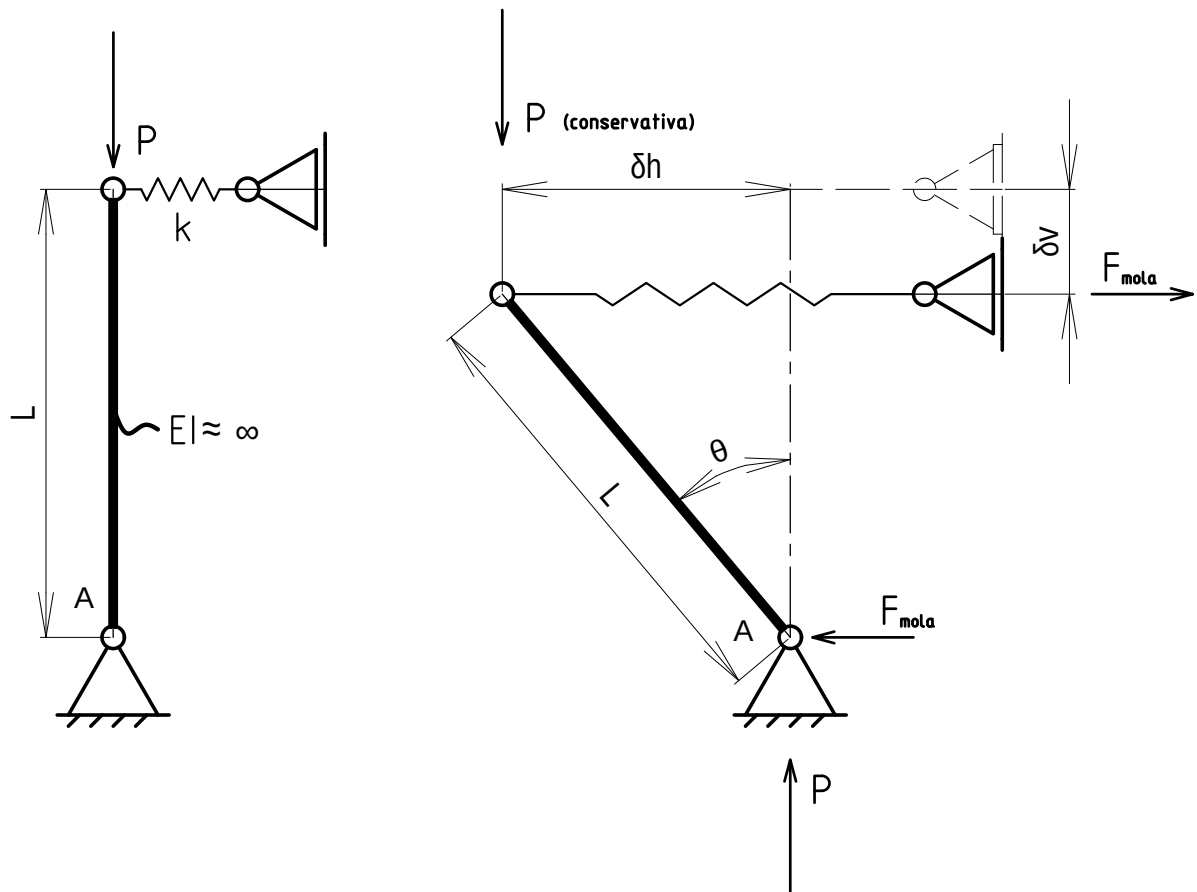
$$P_{cr} = \lim_{\theta \rightarrow 0} \frac{k_{\text{mola}}}{L} \cdot \frac{\theta}{\sin \theta} \Rightarrow \boxed{P_{cr} = \frac{k_{\text{mola}}}{L}} \quad (1.4)$$

2 TRABALHO PRÁTICO 02

Uma barra rígida biapoada possui apoio elástico de translação no topo. Pede-se traçar os gráficos de força aplicada (P) versus deslocamento horizontal (δ_H) e vertical (δ_V) referentes ao ponto de aplicação da força. Adotar:

$$\begin{cases} k_{\text{mola}} = 1 \text{ kN/cm} \\ L = 100 \text{ cm} \\ \Delta\theta = \frac{\pi}{90} \text{ rad} \end{cases}$$

Figura 3: Sistema estrutural do Trabalho Prático 02.



Neste sistema estrutural, assume-se, novamente, relação constitutiva linear da mola:

$$F_{\text{mola}} = k_{\text{mola}} \cdot \delta_h \quad (2.1)$$

E como relações cinemáticas, tem-se a associação entre as variações δ_h e δ_v com o ângulo de giro θ da barra por meio da Equação 2.2.

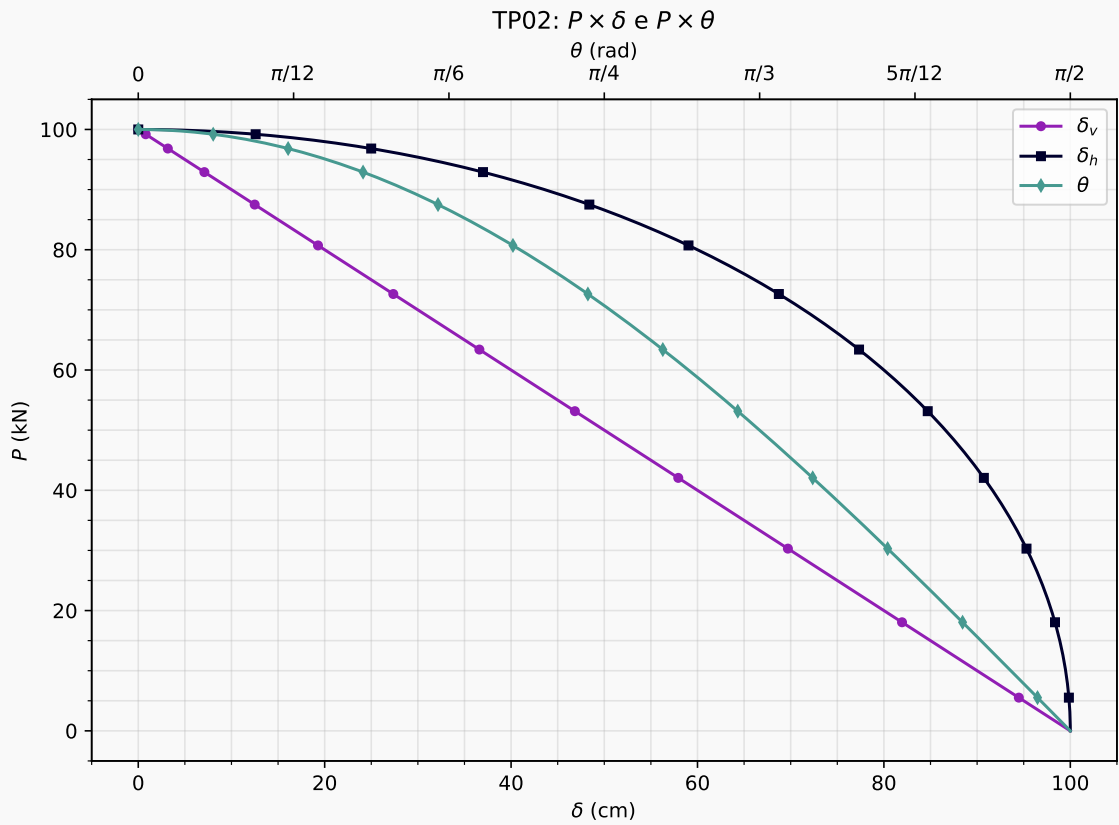
$$\begin{cases} \delta_h = L \sin \theta \\ \delta_v = L (1 - \cos \theta) \end{cases} \quad (2.2)$$

Assim, o equilíbrio de momentos em relação ao apoio fixo A resultam na carga P em função do ângulo θ dado na Equação 2.3, assumindo grandes deslocamentos.

$$P = k_{\text{mola}} L \cdot \cos \theta \quad (2.3)$$

Graficamente, essa relação não-linear é expressa pelas curvas $P \times \delta_v$ e $P \times \delta_h$ mostradas na Figura 4, geradas pelo código dado em Apêndice A.3.

Figura 4: TP02 – Curvas do carregamento vertical P .



Nesse sistema estrutural, a carga crítica se relaciona com as constantes do problema da seguinte forma:

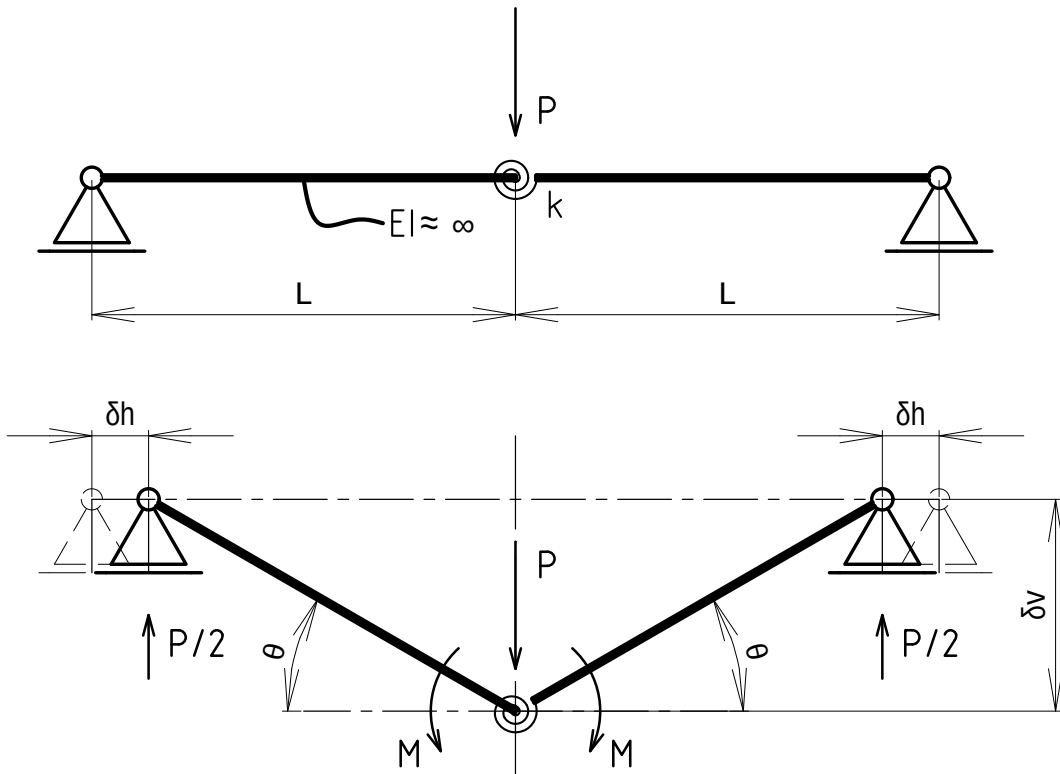
$$P_{cr} = \lim_{\theta \rightarrow 0} k_{\text{mola}} L \cdot \cos \theta \Rightarrow P_{cr} = k_{\text{mola}} L \quad (2.4)$$

3 TRABALHO PRÁTICO 03

Fazer os gráficos $P \times \delta_V$, $P \times \delta_H$, $P \times \theta$ para $\theta \in [0, \frac{\pi}{2}]$. Adotar:

$$\begin{cases} k &= 1000 \text{ kN} \cdot \text{cm/rad} \\ L &= 100 \text{ cm} \\ \Delta\theta &= \frac{\pi}{36} \text{ rad} \end{cases}$$

Figura 5: Sistema estrutural do Trabalho Prático 03.



Na situação deformada, os variacionais δ_h e δ_v se relacionam com o ângulo θ a partir das relações cinemáticas dadas na Equação 3.1. O sistema é simétrico em relação ao ponto de conexão das barras com a mola de torção, o que implica na equivalência dos ângulos de giro das duas barras.

$$\begin{cases} \delta_h &= L(1 - \cos \theta) \\ \delta_v &= L \sin \theta \end{cases} \quad (3.1)$$

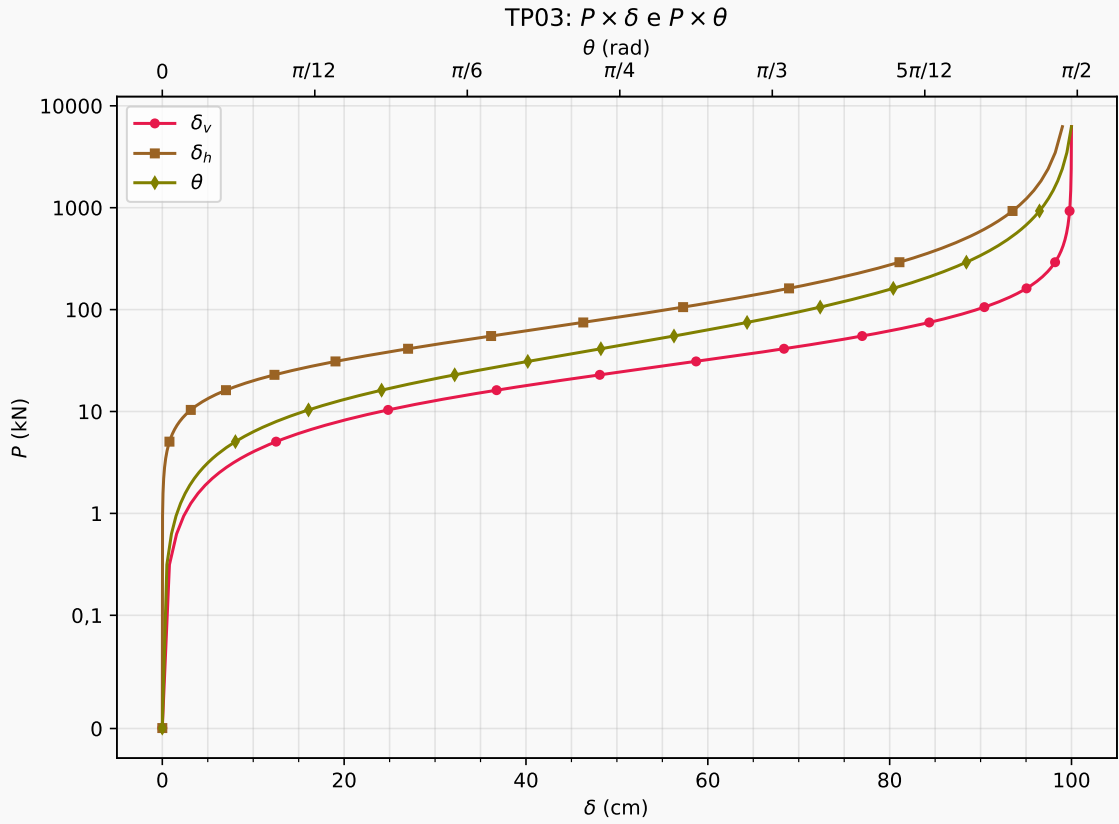
Sendo assim, no equilíbrio, a carga P se relaciona com o valor de θ a partir da Equação 3.3, assumindo relação constitutiva linear da mola de torção (Equação 3.2).

$$M = k_{\text{mola}} \cdot (2\theta) \quad (3.2)$$

$$P = \frac{4k_{\text{mola}}}{L} \cdot \frac{\theta}{\cos \theta} \quad (3.3)$$

Essa relação não-linear é expressa pelas curvas $P \times \theta$, $P \times \delta_v$ e $P \times \delta_h$ mostradas na Figura 6, geradas pelo código dado em Apêndice A.4.

Figura 6: TP03 – Curvas do carregamento vertical P .



Algo interessante ocorre quando se calcula a carga crítica desse sistema estrutural: o valor dá zero, indicando se tratar de um mecanismo rotulado rígido, que não se autosustenta.

$$P_{cr} = \lim_{\theta \rightarrow 0} \frac{4k_{\text{mola}}}{L} \cdot \frac{\theta}{\cos \theta} \Rightarrow \boxed{P_{cr} = 0} \quad (3.4)$$

4 TRABALHO PRÁTICO 04

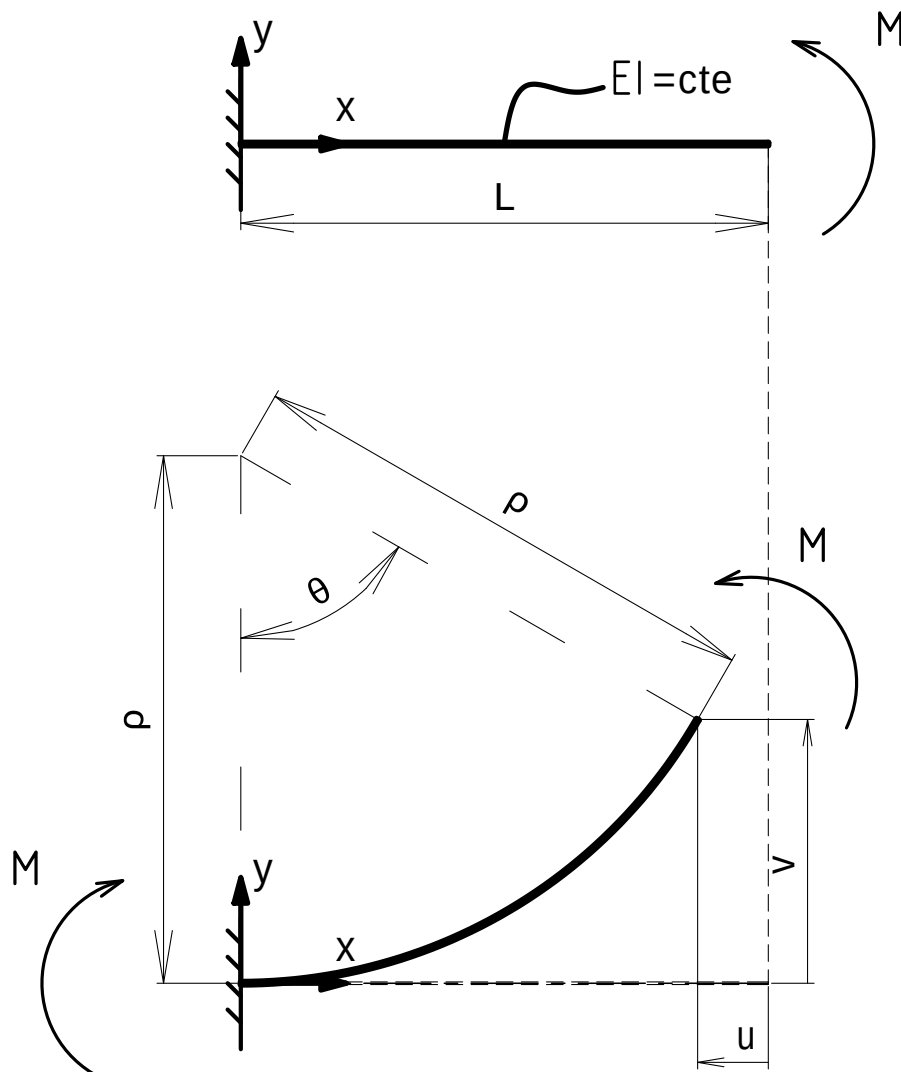
Para a viga engastada com momento constante aplicado na extremidade livre pede-se:

- Traçar os gráficos $M \times \theta$, $M \times u$, $M \times v$ para $\pi \in [0, 2\pi]$ rad no ponto $x = L$
- Detalhar a solução da viga deformada para $\theta = \pi$ e $\theta = 2\pi$, considerando-se 11 pontos igualmente espaçados (10 elementos) ao longo do comprimento da viga.

Adotar:

$$\begin{cases} EI = 4 \times 10^4 \text{ N/cm} \\ L = 500 \text{ cm} \\ \Delta\theta = \frac{\pi}{36} \text{ rad} \end{cases}$$

Figura 7: Sistema estrutural do Trabalho Prático 04.



A cinemática do problema se baseia nas relações trigonométricas entre deslocamentos lineares na extremidade livre, horizontal (u) e vertical (v), e raio de curvatura da viga na configuração deformada, ρ , além da relação diferencial entre o ângulo θ e v , todas elas agrupadas na Equação 4.1.

$$\begin{cases} u &= L - \rho \cdot \sin \theta \\ v &= \rho (1 - \cos \theta) \\ \theta &= \frac{dv}{dx} = v' \end{cases} \quad (4.1)$$

Da teoria de vigas de Euler-Bernoulli, o momento fletor M em uma viga engastada-livre se relaciona com a curvatura ρ a partir da relação constitutiva dada pela Equação 4.2.

$$M = \frac{1}{\rho} \cdot EI \longrightarrow \rho = \frac{EI}{M} \quad (4.2)$$

O desenvolvimento na situação de equilíbrio prossegue com a solução do problema de valor inicial que relaciona a linha neutra com o momento fletor ao longo do eixo x (longitudinal) da viga engastada-livre:

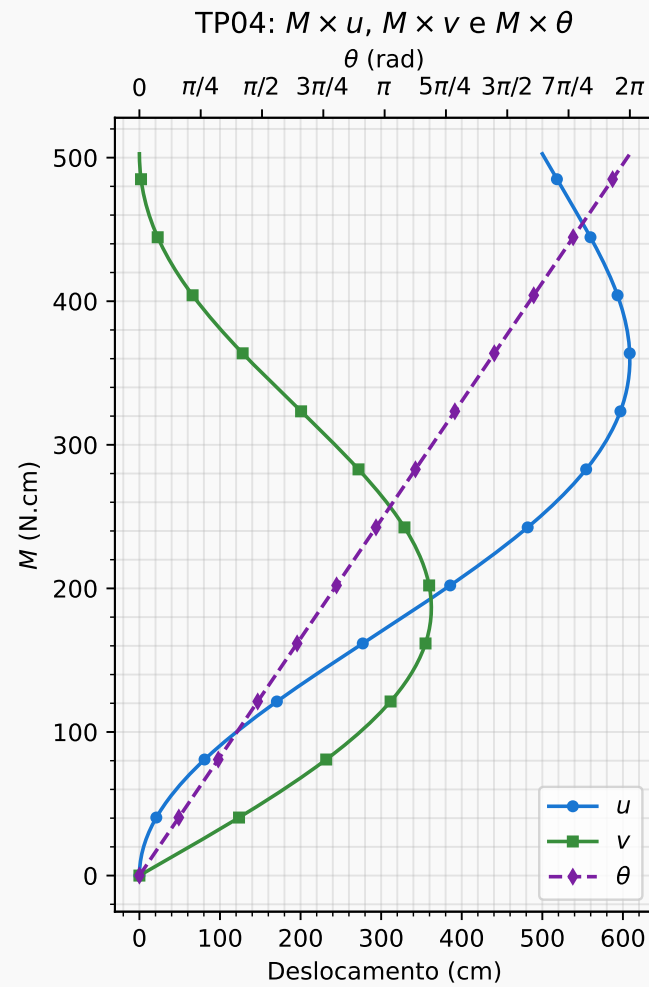
$$\begin{cases} M(x) &= EI \frac{d^2v}{dx^2} \\ v(x=0) &= 0 \\ v'(x=0) &= 0 \end{cases} \quad (4.3)$$

Cuja solução implica no equacionamento de v e θ em função de x :

$$\begin{cases} v(x) &= \frac{M}{2EI} x^2 \\ \theta(x) &= \frac{M}{EI} x \end{cases} \quad (4.4)$$

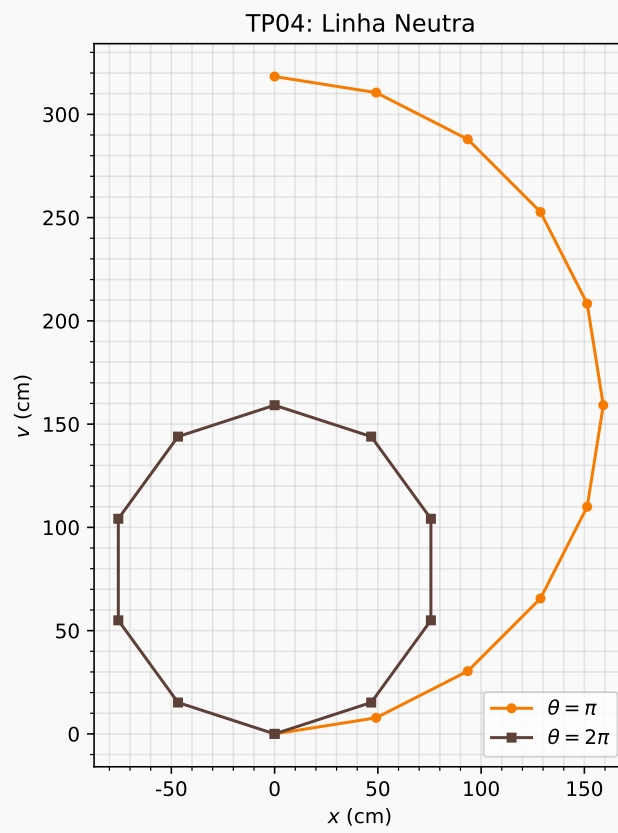
As quais geram relações diretas entre o momento M e rigidez à flexão EI da viga quando substituídas na Equação 4.1.

- a) Os gráficos de $M \times \theta$, $M \times u$ e $M \times v$ foram gerados pelo código mostrado no Apêndice A.5, incluídos aqui na Figura 8.

Figura 8: TP04 – Curvas do momento fletor M .

- b) Para $\theta = \pi$ e $\theta = 2\pi$, a linha neutra se deforma conforme apresentado na Figura 9, dada a teoria exposta acima.

Figura 9: TP04 – Linha neutra $v(x)$ para $\theta = \pi$ e para $\theta = 2\pi$.

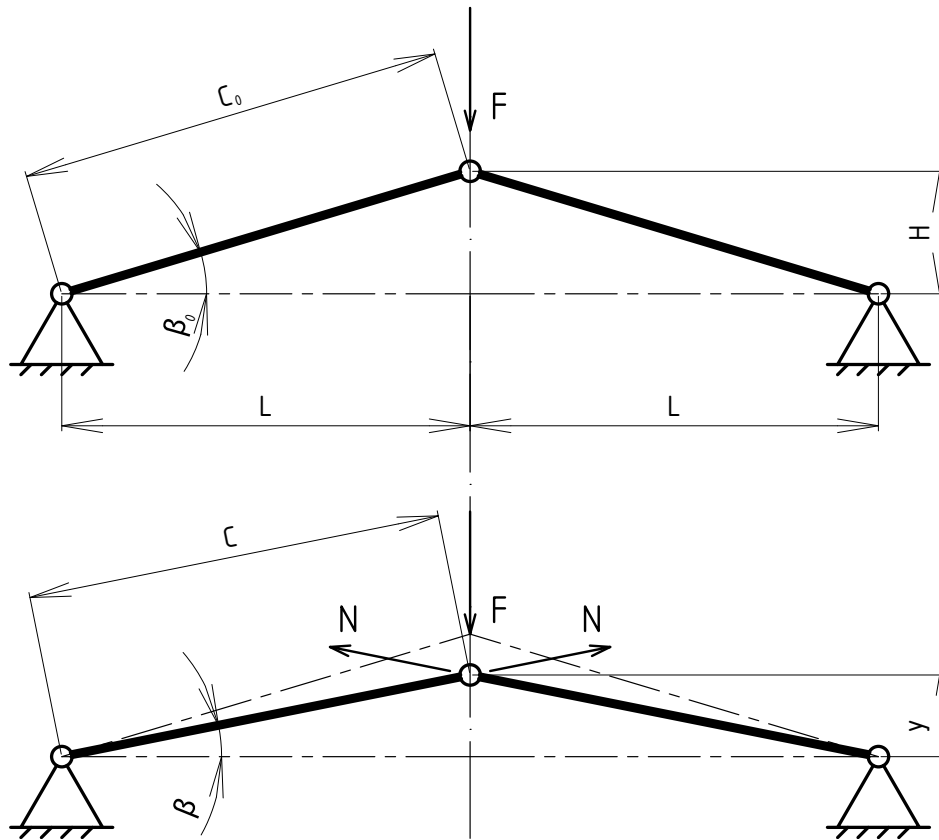


5 TRABALHO PRÁTICO 05

Fazer os gráficos $F \times \delta_v$ (deslocamento vertical) e $N \times \delta_v$ para $y \in [0, 40; 0, 20]$ m.
Adotar:

$$\begin{cases} \Delta y &= -0,01 \text{ m} \\ EA_0 &= 133,865 \text{ kN} \\ L &= 4,0 \text{ m} \\ H &= 0,20 \text{ m} \end{cases}$$

Figura 10: Sistema estrutural do Trabalho Prático 05.



A compatibilidade cinemática do problema se desenvolve em duas três etapas:

1. Configuração inicial – relação trigonométrica entre c_0 e L

$$c_0 = \frac{L}{\cos \beta_0} \quad (5.1)$$

2. Configuração deformada – relação trigonométrica entre c e L

$$c = \frac{L}{\cos \beta} \quad (5.2)$$

3. Deslocamento compressivo de cada barra

$$\Delta c = c - c_0 = L (\sec \beta - \sec \beta_0) \quad (5.3)$$

Sendo que os ângulos β_0 e β se relacionam com a posição vertical do nó central de conexão entre as barras, inicial (H) e final (y), a partir da Equação 5.4.

$$\begin{cases} \beta_0 &= \tan^{-1} \left(\frac{H}{L} \right) \\ \beta &= \tan^{-1} \left(\frac{y}{L} \right) \end{cases} \quad (5.4)$$

Agora, na situação de equilíbrio na configuração deformada da treliça, as componentes horizontais das forças nomais N das barras se cancelam, sobrando apenas as componentes verticais, equilibradas com a força externa F . Essas duas forças se relacionam entre si com o ângulo β , como mostra a Equação 5.5.

$$F = 2N \sin \beta \quad (5.5)$$

Por fim, assumindo relação constitutiva do tipo Hookeana para o par tensão-deformação atuante nas barras, i.e., $\sigma = E\varepsilon$, tem-se que

$$\frac{N}{A_0} = E \cdot \frac{\Delta c}{c_0} = E \cdot \left(\frac{c}{c_0} - 1 \right) \quad (5.6)$$

Substituindo as relações cinemáticas e a relação de equilíbrio na Equação 5.6, chega-se numa relação direta entre a força F e os ângulos β_0 e β :

$$F = 2EA_0 \tan \beta (\cos \beta_0 - \cos \beta) \quad (5.7)$$

É de interesse para a descrição da implementação numérica adotada (Apêndice A.6) a enumeração dos passos efetuados:

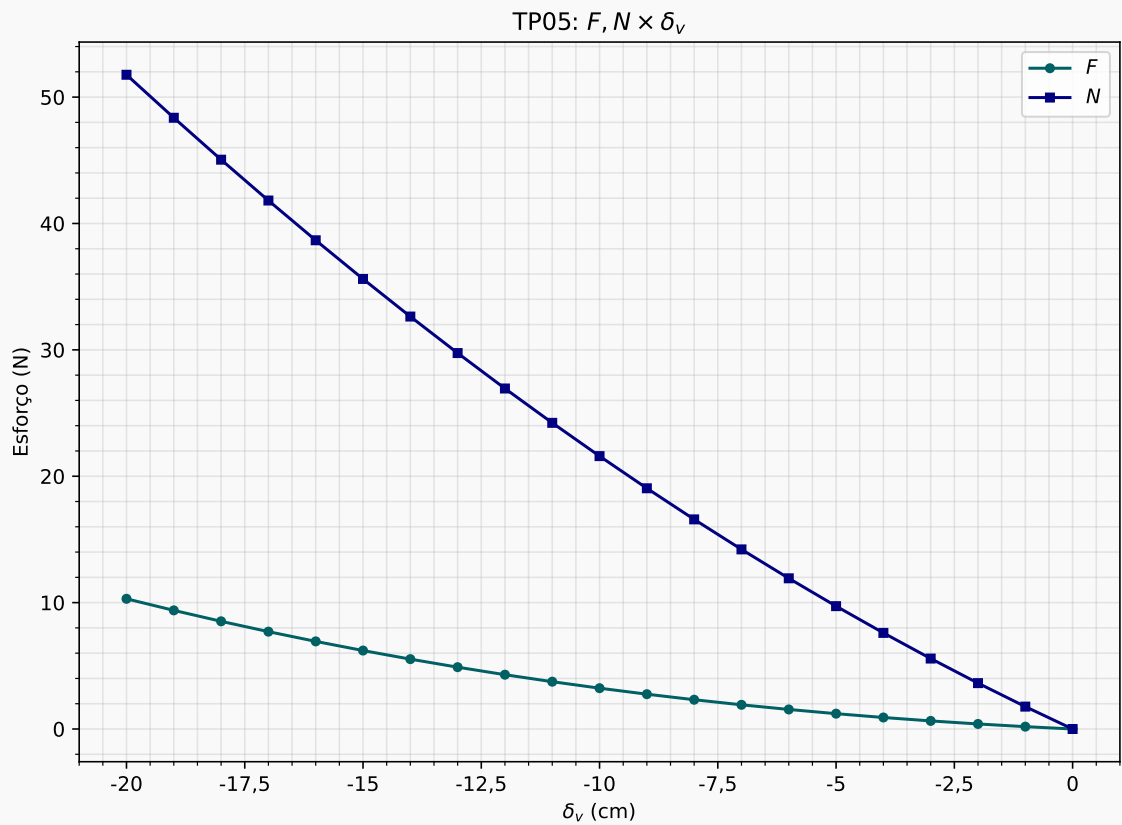
1. Calcular β_0
2. Para dado $y_i = y_{i-1} + \Delta y$, calcular e armazenar:
 - (a) deslocamento vertical $\delta_v = H - y$;
 - (b) ângulo β ;
 - (c) força F ;

(d) força normal N ;

3. Imprimir os gráficos

A Figura 11 ilustra os gráficos $F \times \delta_v$ e $N \times \delta_v$ obtidos no processo, para os dados da treliça de von Mises fornecidos no enunciado.

Figura 11: TP05 – Curvas da treliça de von Mises.



6 TRABALHO PRÁTICO 06

Fazer gráficos de

- a) estiramento (λ) *versus* medidas de deformação clássicas (ϵ^*);
- b) estiramento *versus* conjugados de tensão normalizados $\left(\frac{\sigma^*}{\sigma_B}\right)$;

para $\lambda \in]0; 2]$.

Para a construção dos gráficos requeridos, considera-se a família de Seth-Hill das medidas de deformação e respectivos pares conjugados de tensão normalizados em relação à tensão de Biot σ_B , resumidos na Tabela 1.

Tabela 1: Medidas de deformação clássicas e pares conjugados de tensão em função do estiramento.

Medida de deformação	ϵ^*	$\left(\frac{\sigma^*}{\sigma_B}\right)$
<u>Almansi</u>	$\frac{1}{2} \left(1 - \frac{1}{\lambda^2}\right)$	λ^3
<u>Hiperbólica</u>	$1 - \frac{1}{\lambda}$	λ^2
<u>Logarítmica</u>	$\ln(\lambda)$	λ
<u>Biot</u>	$\lambda - 1$	1
<u>Green</u>	$\frac{1}{2} (\lambda^2 - 1)$	$\frac{1}{\lambda}$

As curvas de medidas de deformação estão plotadas na Figura 12, e dos conjugados de tensão normalizados, na Figura 13.

Figura 12: TP06 – Medidas clássicas de deformação em função do estiramento.

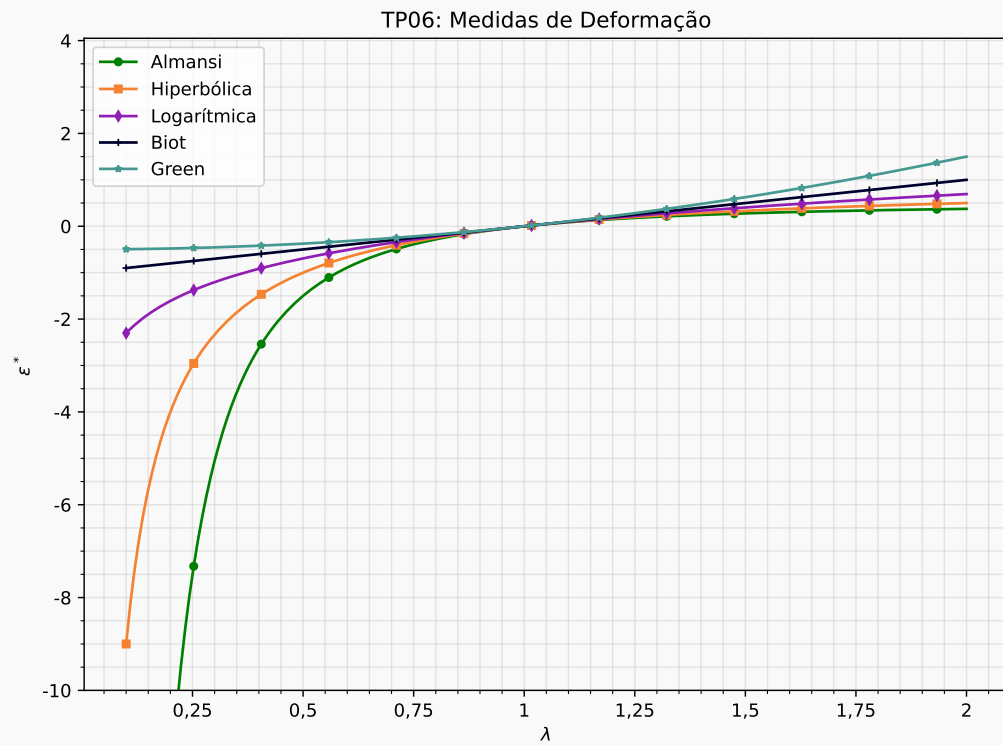
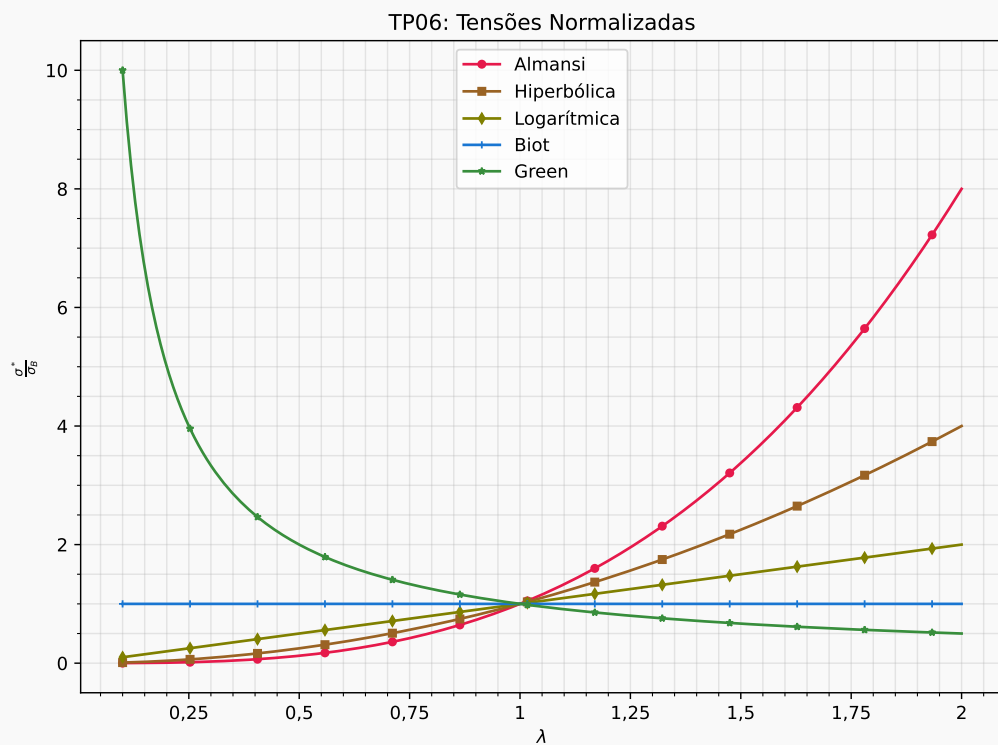


Figura 13: TP06 – Conjugados de tensão normalizados em função do estiramento.



7 TRABALHO PRÁTICO 07

1. Calcular as energias de deformação específicas para as medidas de deformação Logarítmica e de Green.
2. Encontrar os valores das constantes para leis do tipo: $\sigma^* = E \cdot \varepsilon^*$ (leis Hookeanas).

A energia de deformação específica u^* se relaciona com o estiramento λ a partir da Equação 7.1.

$$u^* = \int_{\varepsilon^*} \sigma^* d\varepsilon^* = \int_{\lambda} \left(\sigma^* \frac{\partial \varepsilon^*}{\partial \lambda} \right) d\lambda \quad (7.1)$$

Em consequência, considerando lei constitutiva Hookeana para os pares de tensão-deformação, a expressão para o cálculo da energia de deformação específica se transforma em:

$$u^* = E \int_{\lambda} \left(\varepsilon^* \frac{\partial \varepsilon^*}{\partial \lambda} \right) d\lambda \quad (7.2)$$

Ou, então, numa relação direta com a deformação ε^* , dada na Equação 7.3, que será utilizada nesta resolução.

$$u^* = E \int_{\varepsilon^*} (E \cdot \varepsilon^*) \varepsilon^* d\varepsilon^* \longrightarrow \boxed{u^* = \frac{E}{2} [\varepsilon^*(\lambda)]^2} \quad (7.3)$$

1. Energia de deformação específica

A medida de deformação Logarítmica $\varepsilon^* = \varepsilon_L$ é expressa por:

$$\varepsilon_L = \ln \lambda \quad (7.4)$$

Implicando que a energia de deformação para a medida de deformação Logarítmica é dada por:

$$\boxed{u_L = \frac{E}{2} (\ln \lambda)^2} \quad (7.5)$$

Já a medida de deformação de Green se expressa na Equação 7.6, permitindo desenvolver a expressão para a energia de deformação específica para esta medida de deformação (u_G) na Equação 7.7.

$$\varepsilon_G = \frac{1}{2} (\lambda^2 - 1) \quad (7.6)$$

$$\Rightarrow u_G = \frac{E}{2} \left[\frac{1}{2} (\lambda^2 - 1) \right]^2 \longrightarrow \boxed{u_G = \frac{E \lambda^2}{8} (\lambda^2 - 2) + \frac{E}{8}} \quad (7.7)$$

2. Constantes para leis do tipo Hookeanas

Os conjugados de tensão de cada medida da deformação clássica se relacionam com a tensão de Biot σ_B , dada na Equação 7.8, a partir de expressões envolvendo o estiramento.

$$\sigma_B = E (\lambda - 1) \quad (7.8)$$

Para as medidas de deformação Logarítmica e de Green, essas relações são mostradas na Equação 7.9.

$$\begin{cases} \sigma_L = \lambda \sigma_B = E \cdot (\lambda^2 - \lambda) \\ \sigma_G = \frac{1}{\lambda} \sigma_B = E \cdot \left(1 - \frac{1}{\lambda}\right) \end{cases} \quad (7.9)$$

Além disso, considerando a constante de proporcionalidade E_L para a medida de deformação Logarítmica e E_G para a medida de deformação de Green, as leis constitutivas equivalente são expressas como:

$$\begin{cases} \sigma_L = E_L \cdot \varepsilon_L = E_L \cdot \ln \lambda \\ \sigma_G = E_G \cdot \varepsilon_G = E_G \cdot \frac{1}{2} (\lambda^2 - 1) \end{cases} \quad (7.10)$$

Portanto, comparando-se a Equação 7.10 com a Equação 7.9, encontram-se relações entre as constantes de proporcionalidade e o módulo de Young E referente à medida de deformação de Biot – a qual está ligada a ensaios de caracterização que medem a deformação com base nas dimensões iniciais. Para a medida de deformação Logarítmica, tem-se, pois, que:

$$\frac{E_L}{E} = \frac{\lambda^2 - \lambda}{\ln \lambda} \quad (7.11)$$

Enquanto, para a medida de deformação de Green, a razão entre E_G e E se torna:

$$\frac{E_G}{E} = \frac{2}{\lambda^2 + \lambda} \quad (7.12)$$

Retomando esse resultado na Equação 7.9, as leis constitutivas do tipo Hookeanas

para as medidas de deformação Logarítmica e de Green são formuladas na Equação 7.13 e Equação 7.14, respectivamente.

$$\sigma_L = \frac{\lambda^2 - \lambda}{\ln \lambda} E \cdot \varepsilon_L \quad (7.13)$$

$$\sigma_G = \frac{2E}{\lambda^2 + \lambda} \cdot \varepsilon_G \quad (7.14)$$

8 TRABALHO PRÁTICO 08

Para a treliça espacial simétrica de 3 barras, pede-se fazer gráficos $P \times \delta_v$ (deslocamento vertical) e $P \times N$ (força axial nas barras) considerando-se $y \in [-4; 2]$ m usando as cinco leis constitutivas que relacionam linearmente as medidas de deformação clássicas e seus respectivos pares conjugados de tensão. Adotar:

$$\begin{cases} EA_0 &= 133,865 \text{ kN} \\ H &= 200 \text{ cm} \\ L &= 500 \text{ cm} \\ \Delta y &= -0,02 \text{ m} \end{cases}$$

Figura 14: Sistema estrutural do Trabalho Prático 08 – Vista superior

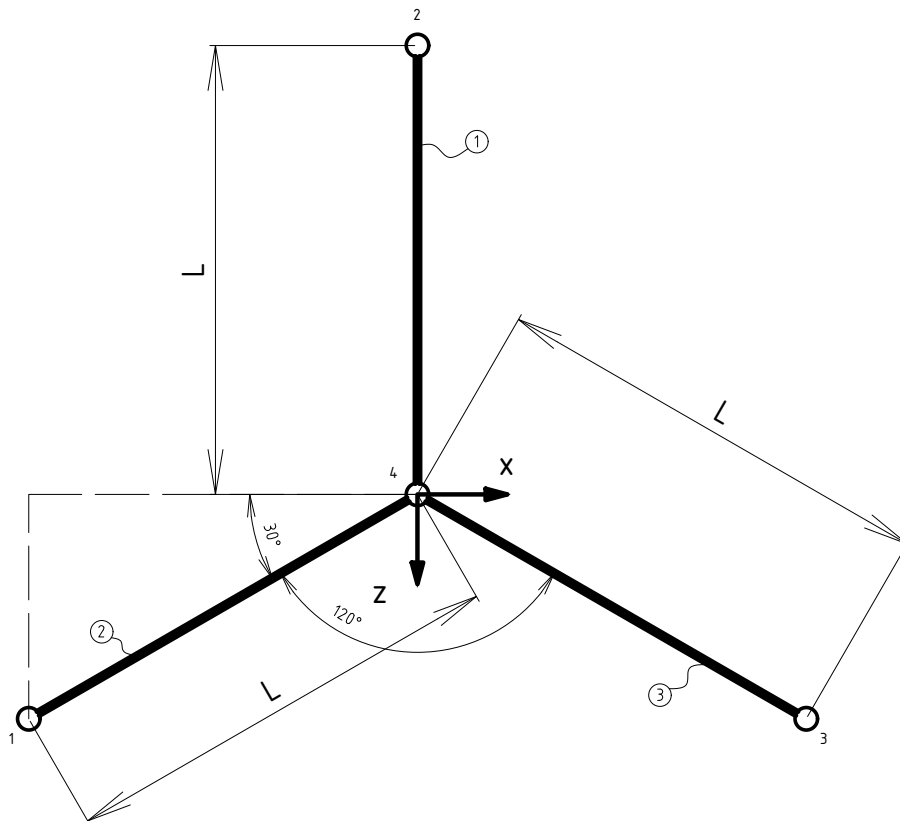
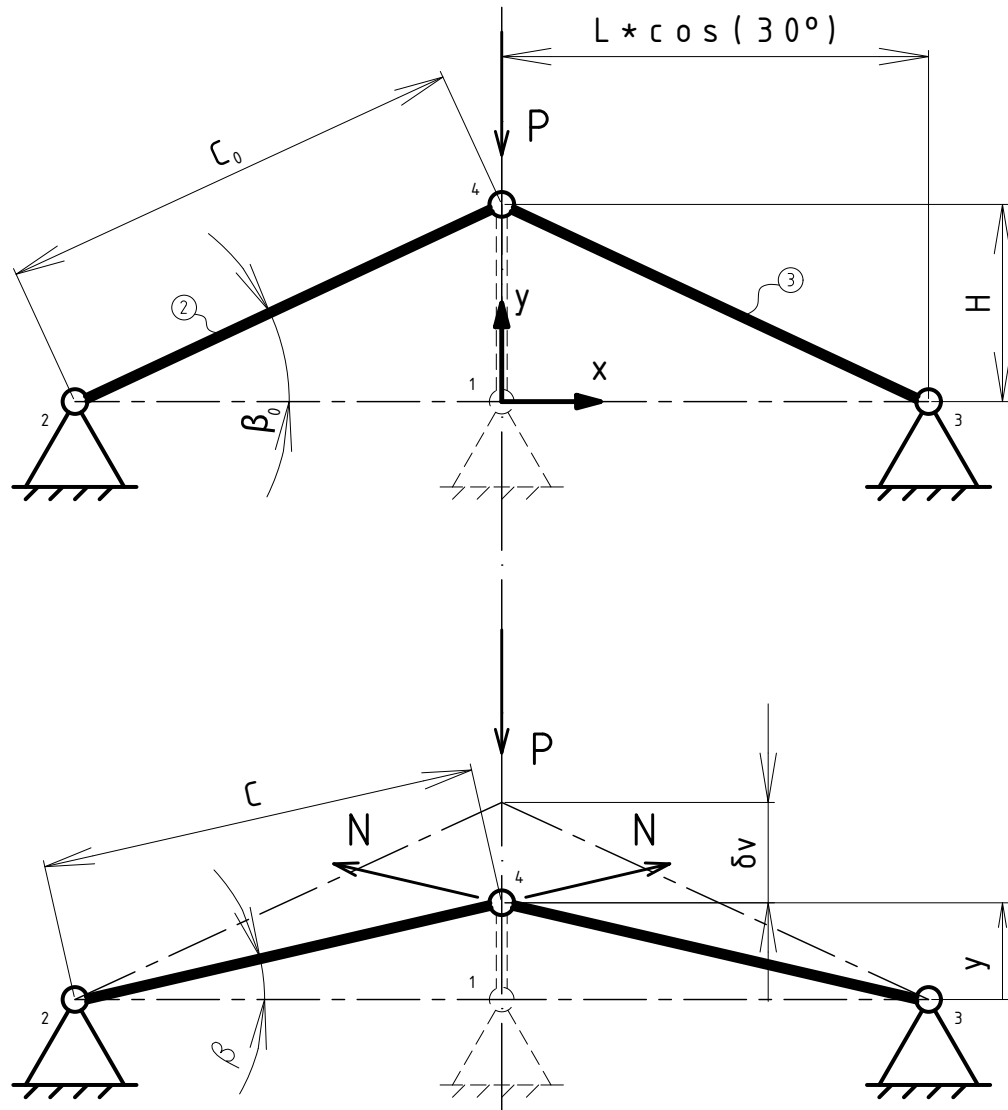


Figura 15: Sistema estrutural do Trabalho Prático 08 – Vista lateral



A treliça espacial do problema possui simetria radial, de modo que a força normal $\vec{N} = (N_x \hat{i} + N_y \hat{j} + N_z \hat{k})$, atuante na direção axial de cada barra, possui o mesmo módulo para todas as barras. No equilíbrio, a resultante nas direções x e z das forças normais se cancelam, de modo que a equação de equilíbrio se torna:

$$3N_y - P = 0 \quad (8.1)$$

Denotando por $\vec{v}_1$ o vetor direção da barra ① na configuração deformada, α o ângulo entre o vetor normal $\vec{N}$ e a vertical, e $N = ||\vec{N}||$, tem-se que:

$$N_y = N \cos \alpha \quad (8.2)$$

$$\cos \alpha = \frac{\langle \vec{v}_1, y \hat{j} \rangle}{\|\vec{v}_1\| \cdot \|y \hat{j}\|} = \frac{(0, y, L) \cdot (0, y, 0)}{\sqrt{y^2 + L^2} \cdot y} \rightarrow \cos \alpha = \frac{y}{\sqrt{y^2 + L^2}} \quad (8.3)$$

Por simetria, o ângulo α será igual para todas as barras. Assim, a equação final do equilíbrio é dada por

$$N = -\frac{P}{3 \cos \alpha} \quad (8.4)$$

Além disso, as medidas de deformação clássicas (família Seth-Hill) se baseiam no estiramento λ da barra, no caso 1D, o qual é calculado como a razão entre o comprimento c da barra na configuração deformada e o comprimento c_0 da barra na configuração inicial:

$$\lambda = \frac{c}{c_0} \quad (8.5)$$

Dimensões essas que se relacionam com os ângulos β e β_0 baseando-se na compatibilidade cinemática do problema, dada na Equação 8.6.

$$\left\{ \begin{array}{l} \tan \beta_0 = \frac{H}{L} \\ \tan \beta = \frac{y}{L} \\ c_0 = L \sec \beta_0 = \sqrt{L^2 - H^2} \\ c = L \sec \beta = \sqrt{L^2 - y^2} \end{array} \right. \quad (8.6)$$

Dessa forma, o estiramento se relaciona com os ângulos β e β_0 de forma não-linear, como mostrado na Equação 8.7.

$$\lambda = \frac{c}{c_0} = \frac{L \sec \beta}{L \sec \beta_0} \rightarrow \lambda = \frac{\cos \beta_0}{\cos \beta} \quad (8.7)$$

Ou, em termos das dimensões L , H e y :

$$\lambda = \sqrt{\frac{1 - \left(\frac{y}{L}\right)^2}{1 - \left(\frac{H}{L}\right)^2}} \quad (8.8)$$

Enquanto o conjugado de tensão de cada medida de deformação se relaciona com a tensão de Biot σ_B , a qual vale:

$$\sigma_B = \frac{N}{A_0} \quad (8.9)$$

Abaixo, serão construídas relações entre a carga P e a coordenada y do nó 4 (nó central) para cada uma das cinco relações constitutivas da família de Seth-Hill.

a) Green ($m = 2$)

$$\begin{aligned}\sigma_G &= \frac{1}{\lambda} \cdot \sigma_B = E \cdot \varepsilon_G \\ \frac{1}{\lambda} \cdot \frac{N}{A_0} &= E \cdot \frac{1}{2} (\lambda^2 - 1) \\ -\frac{P}{\lambda \cdot 3A_0 \cos \alpha} &= \frac{E}{2} (\lambda^2 - 1)\end{aligned}$$

$$\boxed{P = -\frac{3EA_0 \cos \alpha}{2} \cdot \lambda (\lambda^2 - 1)} \quad (8.10)$$

b) Biot ($m = 1$):

$$\begin{aligned}\sigma_B &= E \cdot \varepsilon_B \\ \frac{N}{A_0} &= E \cdot (\lambda - 1) \\ -\frac{P}{3A_0 \cos \alpha} &= E (\lambda - 1)\end{aligned}$$

$$\boxed{P = -3EA_0 \cos \alpha (\lambda - 1)} \quad (8.11)$$

c) Logarítmica ($m = 0$)

$$\begin{aligned}\sigma_L &= \lambda \cdot \sigma_B = E \cdot \varepsilon_L \\ \lambda \cdot \frac{N}{A_0} &= E \cdot \ln \lambda \\ -\frac{\lambda \cdot P}{3A_0 \cos \alpha} &= E \cdot \ln \lambda\end{aligned}$$

$$\boxed{P = -3EA_0 \cos \alpha \cdot \frac{\ln \lambda}{\lambda}} \quad (8.12)$$

d) Hiperbólica ($m = -1$)

$$\sigma_H = \lambda^2 \cdot \sigma_B = E \cdot \varepsilon_H$$

$$\lambda^2 \cdot \frac{N}{A_0} = E \cdot \left(1 - \frac{1}{\lambda}\right)$$

$$-\frac{\lambda^2 \cdot P}{3A_0 \cos \alpha} = E \cdot \left(1 - \frac{1}{\lambda}\right)$$

$$\boxed{P = -3EA_0 \cos \alpha \cdot \frac{\lambda - 1}{\lambda^3}} \quad (8.13)$$

e) Almansi ($m = -2$)

$$\sigma_A = \lambda^3 \cdot \sigma_B = E \cdot \varepsilon_A$$

$$\lambda^3 \cdot \frac{N}{A_0} = E \cdot \left(1 - \frac{1}{\lambda^2}\right)$$

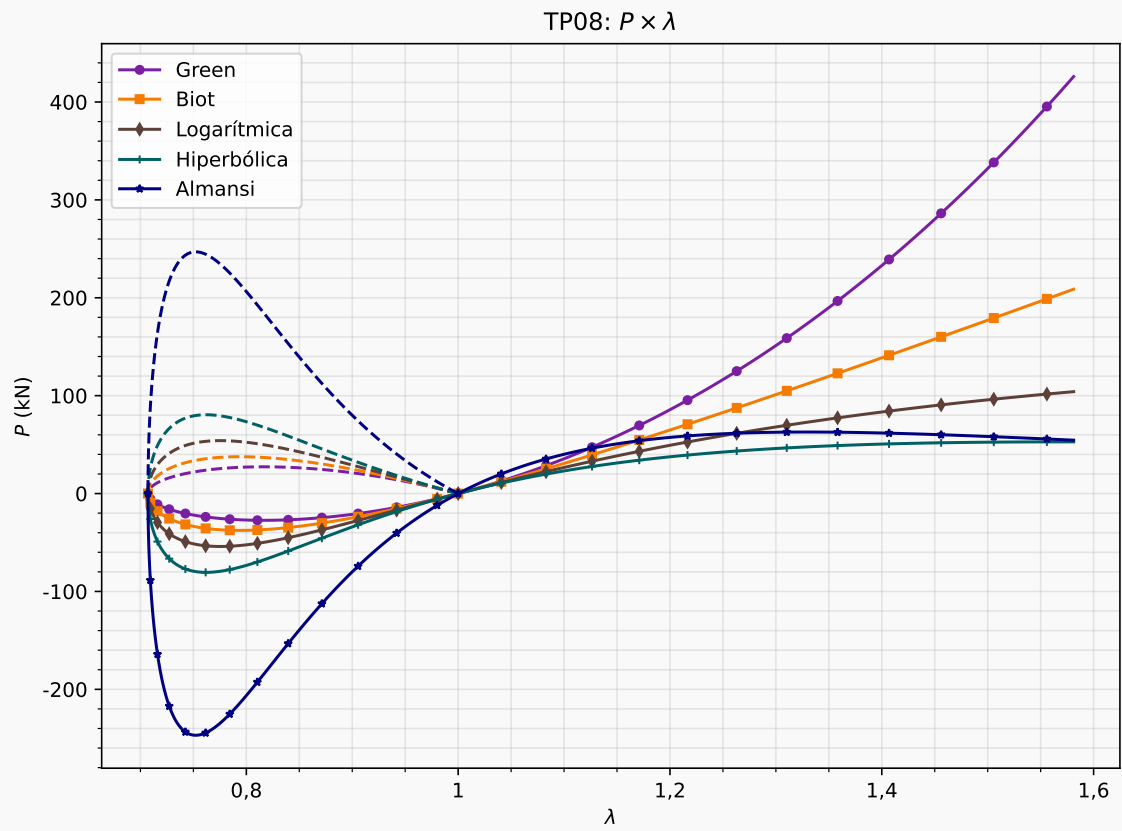
$$-\frac{\lambda^3 \cdot P}{3A_0 \cos \alpha} = E \cdot \left(1 - \frac{1}{\lambda^2}\right)$$

$$\boxed{P = -3EA_0 \cos \alpha \cdot \frac{\lambda^2 - 1}{\lambda^5}} \quad (8.14)$$

Para a implementação numérica, seguiu-se o passo a passo abaixo:

1. Calcular $\cos \beta_0$;
2. Dado um $y_i = y_{i-1} + \Delta y$, calcular e armazenar:
 - (i) deslocamento vertical $\delta_v = H - y$;
 - (ii) $\cos \alpha$;
 - (iii) constante $k \triangleq 3EA_0 \cos \alpha$;
 - (iv) $\cos \beta$;
 - (v) estiramento λ ;
 - (vi) carga P que equilibra o sistema estrutural não-linear geométrico para cada medida de deformação clássica;
 - (vii) magnitude N do esforço normal;

O código mostrado no Apêndice A.8 segue esta rotina de implementação, gerando os gráficos mostrados na Figura 16.

Figura 16: TP08 – Curvas do carregamento vertical P .

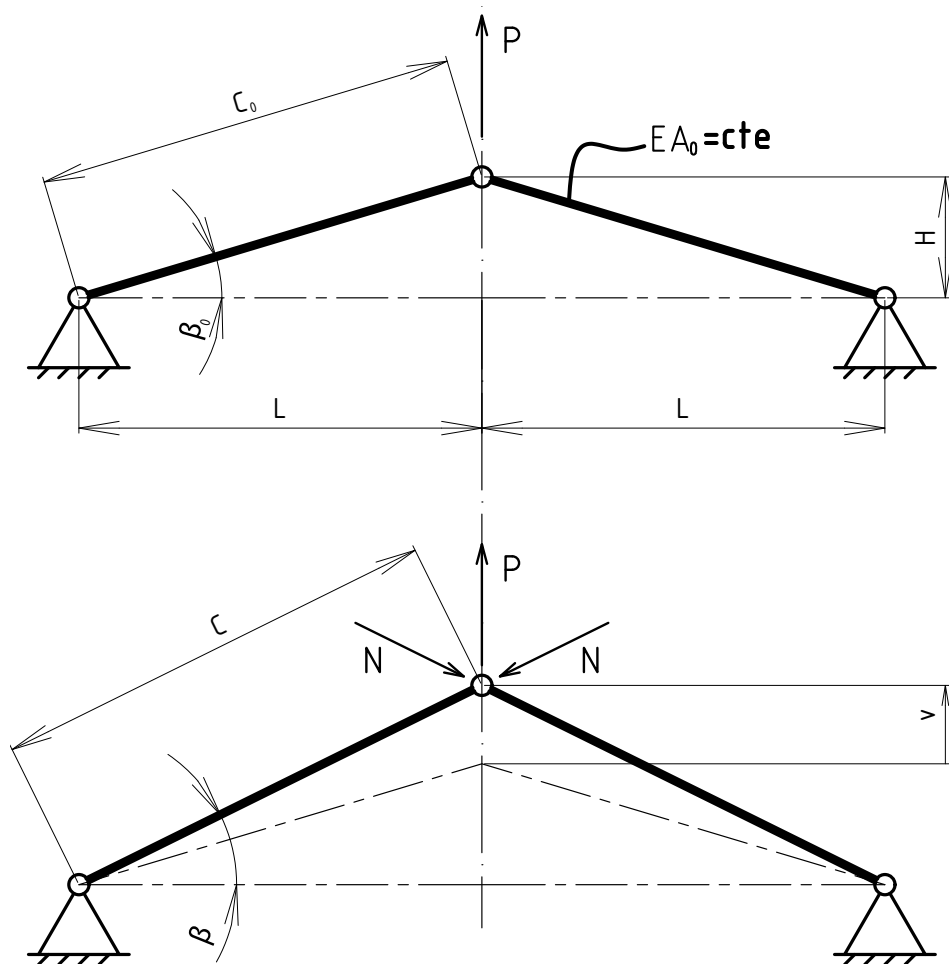
9 TRABALHO PRÁTICO 09

Encontrar o deslocamento v no equilíbrio, considerando-se a medida de deformação de Green e o algoritmo de Newton-Raphson na resolução. Adotar:

$$\begin{cases} E &= 20.000 \text{ kN/cm}^2 \\ L &= 150 \text{ cm} \\ H &= 10 \text{ cm} \\ A_0 &= 5 \text{ cm}^2 \\ P_{\text{ext}} &= 20 \text{ kN} \end{cases}$$

Verificar o valor de deformação encontrado ao final, comparando-a com a resposta linear.

Figura 17: Sistema estrutural do Trabalho Prático 09.



A cinemática do problema se baseia na simples aplicação do Teorema de Pitágoras na configuração inicial, encontrando-se c_0 , e na configuração deformada da treliça,

obtendo-se o valor de c , como detalhado na Equação 9.1.

$$\begin{cases} c_0 &= \sqrt{H^2 + L^2} \\ c &= \sqrt{(H + v)^2 + L^2} \end{cases} \quad (9.1)$$

Implicando no valor de estiramento dado na Equação 9.2.

$$\lambda = \frac{c}{c_0} = \sqrt{1 + \frac{v^2 + 2Hv}{H^2 + L^2}} \quad (9.2)$$

No equilíbrio, as componentes horizontais do esforço normal $\vec{N}$ se anulam, restando apenas o equacionamento do equilíbrio na direção vertical:

$$P = 2N \cdot \frac{H + v}{c} \quad (9.3)$$

Em seguida, considerando lei constitutiva de Saint-Venant-Kirchhoff (medida de deformação de Green), tem-se o seguinte equacionamento:

$$\begin{aligned} \varepsilon_G &= \frac{1}{2} (\lambda^2 - 1) = \left[\left(\frac{c}{c_0} \right)^2 - 1 \right] \\ \varepsilon_G &= \frac{1}{c_0^2} \cdot (v^2 + 2Hv) \end{aligned} \quad (9.4)$$

E conjugados de tensão que se relacionam do seguinte modo:

$$\begin{cases} \sigma_G &= \frac{1}{\lambda} \sigma_B = E_G \cdot \varepsilon_G \\ \sigma_B &= \frac{N}{A_0} \end{cases} \quad (9.5)$$

Assim, retomando o equacionamento do equilíbrio, tem-se que:

$$N = \frac{EA_0}{c_0^3} \cdot c (v^2 + 2Hv) \quad (9.6)$$

Agora, resolvendo-se o problema não-linear a partir do método de Newton-Raphson, é possível obter uma expressão direta para vetor de resíduos g em função do deslocamento vertical v , uma vez que a carga P aplicada em cada passo iterativo e o carregamento P_{ext} imposto ao nó central da treliça, devem ser equivalentes no equilíbrio:

$$\begin{aligned} g &= P - P_{\text{ext}} \\ g &= 2N \cdot \frac{H + v}{c} - P_{\text{ext}} \\ g &= \frac{2(H + v)}{c} \cdot \frac{EA_0}{c_0^3} \cdot c (v^2 + 2Hv) - P_{\text{ext}} \end{aligned}$$

$$g = \frac{EA_0}{c_0^3} (v^3 + 3Hv^2 + 2H^2v) - P_{\text{ext}} \quad (9.7)$$

O gradiente desse vetor de resíduos é então expresso na Equação 9.8, necessário para o procedimento numérico do método de Newton-Raphson.

$$\nabla g = \frac{EA_0}{c_0^3} \cdot (3v^2 + 6Hv + 2H^2) \quad (9.8)$$

Considerando o equacionamento geral do método de Newton-Raphson, a saber:

$$\begin{cases} \Delta^{i+1}u &= -\frac{g(^iu)}{\nabla g(^iu)} \\ ^{i+1}u &= ^iu + \Delta^{i+1}u \end{cases} \quad (9.9)$$

Precisa de uma previsão inicial como primeira abordagem da solução. No caso desta treliça, a previsão inicial do deslocamento vertical vem da resposta linear obtida pelo Princípio dos Trabalhos Virtuais (PTV):

$$\begin{aligned} P - 2N \sin \beta_0 &= 0 \longrightarrow N_{\text{linear}} = \frac{P}{2} \cdot \sqrt{1 + \left(\frac{L}{H}\right)^2} \\ \implies 1 \cdot v &= 2 \int_L \frac{\bar{N}N}{EA_0} dx = 2 \int_L \frac{N^2}{P \cdot EA_0} dx = \frac{2L}{EA_0} \cdot \frac{N^2}{P} \\ \therefore {}^0v &= \frac{PL}{2EA_0} \cdot \left[1 + \left(\frac{L}{H}\right)^2 \right] \end{aligned} \quad (9.10)$$

O que resulta em $v_{\text{linear}} = 3,3900$ cm. Como critério de convergência, considerou-se que

$$|^{i-1}v - ^iv| \geq 0,001 \quad (9.11)$$

Implementado o método como segue no Apêndice A.9, resultando no valor do deslocamento vertical de

$$v = 2,4355 \text{ cm}$$

Um valor 28,16% menor que a resposta linear do sistema estrutural utilizada como chute inicial.

10 TRABALHO PRÁTICO 10

Ler e resumir o artigo:

YANG, Y. -B.; SHIEH, M. -S. (1990)

"Solution method for nonlinear problems with multiple critical points."

AIAA Journal, V. 28, p. 2110-2116.

O artigo de Yang e Shieh (1990) propõe um método aprimorado para a solução de problemas não-lineares geométricos com múltiplos pontos críticos, denominado **Método de Controle de Deslocamento Generalizado** (*Generalized Displacement Control Method* – GDC). O desenvolvimento se insere no contexto de análises estruturais nas quais a curva carga-deslocamento exibe *pontos limite* (onde a rigidez tangente se anula e o caminho de equilíbrio muda de estável para instável) e *pontos de retrocesso* (*snap-back points*), onde o próprio deslocamento de controle inverte de sentido.

Formulação no Espaço de Dimensão $N + 1$

A equação matricial que governa a j -ésima iteração do i -ésimo incremento de um sistema com N graus de liberdade é escrita como:

$$[K]_{j-1}^i \{u\}_j^i = \lambda_j^i \{P\} + \{R\}_{j-1}^i \quad (10.1)$$

Onde $[K]$ é a matriz de rigidez tangente, $\{u\}$ o vetor de incremento de deslocamentos, $\{P\}$ o vetor de carga de referência, λ o parâmetro de incremento de carga e $\{R\}$ o vetor de forças desequilibradas. Como ambos $\{u\}$ e λ são incógnitas, uma equação de restrição adicional é necessária:

$$\langle C \rangle \{u\}_j^i + k \lambda_j^i = H_j^i \quad (10.2)$$

A seleção adequada das constantes $\langle C \rangle$, k e H_j^i na Equação 10.2 é o ponto central que distingue os diferentes métodos de solução. Os autores mostram que, para garantir estabilidade numérica, o parâmetro de incremento de carga λ_j^i deve permanecer limitado durante toda a análise. Por decomposição da matriz de rigidez generalizada $[\bar{K}_1]$ no espaço $N + 1$, obtém-se a expressão geral:

$$\lambda_j^i = \frac{H_j^i - \lambda_1^i \langle u_1 \rangle_1^{i-1} \{u_2\}_j^i}{\lambda_1^i \langle u_1 \rangle_1^{i-1} \{u_1\}_j^i} \quad (10.3)$$

Na qual $\{u_1\}_1^{i-1}$ é o vetor de deslocamentos resultante da primeira iteração do último passo incremental.

Avaliação dos Métodos Existentes

Os autores utilizam a Equação (10.3) como critério analítico para avaliar os métodos clássicos:

- **Newton-Raphson:** mantém λ limitado nos passos iterativos, porém o incremento de deslocamento $\{u\}_j$ pode divergir próximo a pontos limite, pois $\det[K] \rightarrow 0$ nesses pontos.
- **Controle de Deslocamento:** contorna os pontos limite com sucesso, mas falha nos pontos de retrocesso: quando o deslocamento de controle u^c tende a zero, $\lambda_j^i \rightarrow \infty$.
- **Comprimento de Arco (Arc-Length):** pode gerar sinais incorretos para λ nas proximidades de pontos de retrocesso com gradientes acentuados, além de não fornecer de forma automática a informação necessária para inverter a direção de carregamento.
- **Controle de Trabalho:** baseado no Parâmetro de Rigidez Atual (CSP – *Current Stiffness Parameter*), que apresenta descontinuidades próximas a pontos de retrocesso e muda de sinal tanto em pontos limite quanto em pontos de retrocesso, sendo um indicador inadequado para a inversão de direção de carregamento.

Método Proposto: Controle de Deslocamento Generalizado

Adotando $k = 0$ e $\langle C \rangle = \lambda_1^i \langle u_1 \rangle_1^{i-1}$ na equação de restrição geral, obtém-se, para o passo incremental ($j = 1$):

$$\lambda_1^i = \lambda_1^1 \left(\frac{\langle u_1 \rangle_1^1 \{u_1\}_1^1}{\langle u_1 \rangle_1^{i-1} \{u_1\}_1^i} \right)^{1/2} = \lambda_1^1 (\text{GSP})^{1/2} \quad (10.4)$$

E, para as iterações subsequentes ($j \geq 2$):

$$\lambda_j^i = - \frac{\langle u_1 \rangle_1^{i-1} \{u_2\}_j^i}{\langle u_1 \rangle_1^{i-1} \{u_1\}_j^i} \quad (10.5)$$

O **Parâmetro de Rigidez Generalizado** (GSP – *Generalized Stiffness Parameter*) é definido como:

$$\text{GSP} = \frac{\langle u_1 \rangle_1^1 \{u_1\}_1^1}{\langle u_1 \rangle_1^{i-1} \{u_1\}_1^i} \quad (10.6)$$

Onde o numerador representa os deslocamentos do primeiro passo e o denominador os do passo atual. O GSP apresenta as seguintes propriedades fundamentais:

1. Inicia com valor unitário e é nulo nos pontos limite, tal como o CSP;

2. É negativo apenas para os passos imediatamente após os pontos limite – ao contrário do CSP, que muda de sinal também nos pontos de retrocesso –, tornando-o um indicador confiável e exclusivo para a inversão da direção de carregamento;
3. Não apresenta descontinuidades numéricas nas proximidades de pontos de retrocesso, ao contrário do CSP, que exhibe variações abruptas nessas regiões.

Algoritmo

O procedimento numérico do GDC pode ser sintetizado nas seguintes etapas:

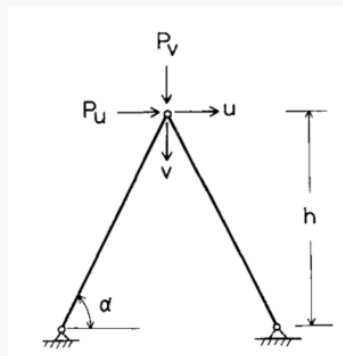
1. Definir o incremento de carga básico λ_1^1 para o início;
2. Para a **primeira iteração** ($j = 1$) de cada passo i : montar $[K]_0^i$; resolver a equação de equilíbrio para $\{u_1\}_1^i$; calcular o GSP (para $i \geq 2$) e determinar λ_1^i ; verificar o sinal do GSP – se negativo, multiplicar λ_1^i por -1 para inverter a direção de carregamento;
3. Para **iterações subsequentes** ($j \geq 2$): calcular as forças desequilibradas; resolver os sistemas para $\{u_1\}_j^i$ e $\{u_2\}_j^i$; determinar λ_j^i e atualizar $\{u\}_j^i$;
4. Atualizar forças internas, nível de carregamento e geometria;
5. Repetir até atingir a convergência e, se o carregamento total não exceder o máximo, avançar ao próximo incremento.

Exemplos Numéricos

Os autores validam o GDC em dois exemplos de referência:

Exemplo 1 – Treliça de von Mises

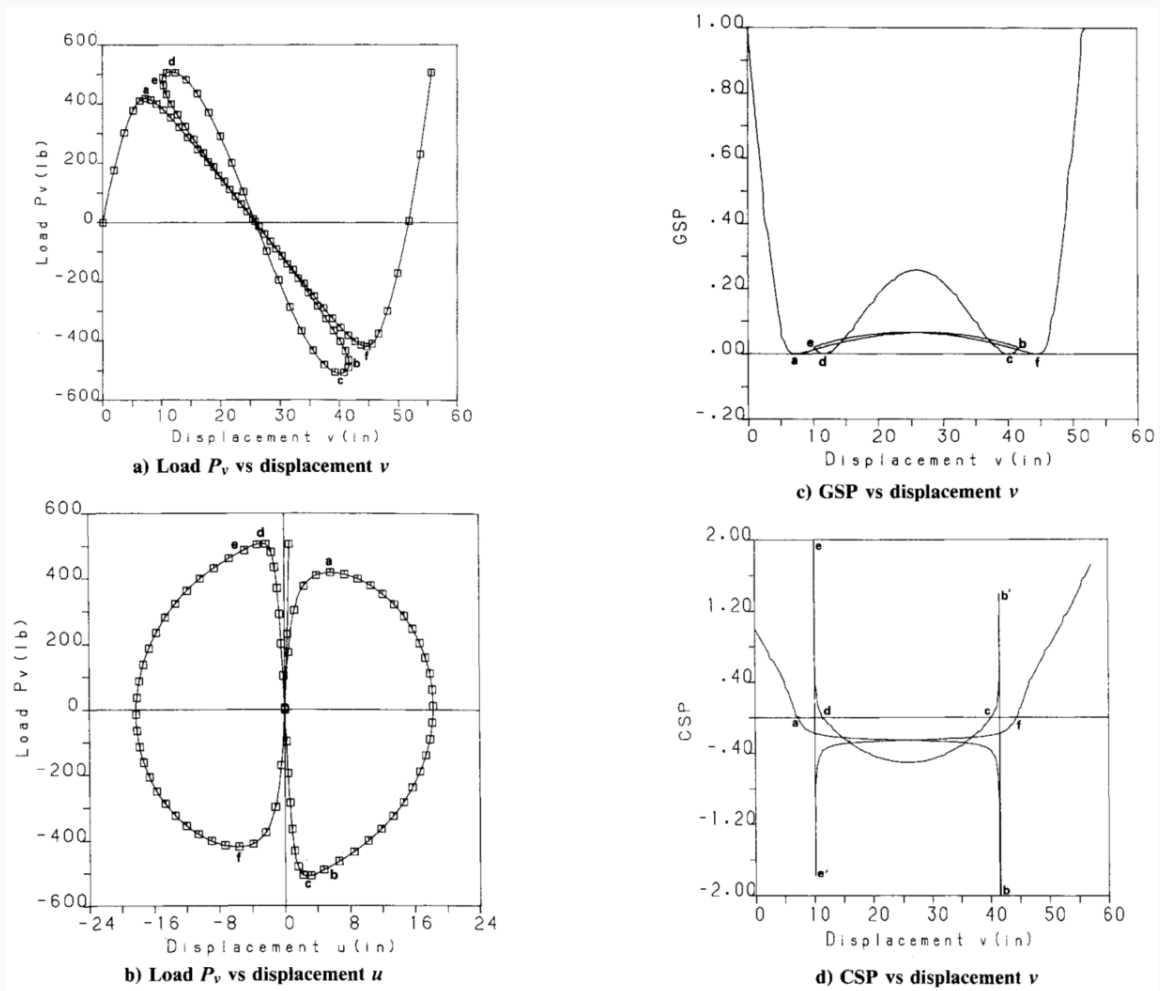
Figura 18: Treliça de dois membros.



Fonte: Adaptado de YANG, Y. -B.; SHIEH, M. -S. (1990)

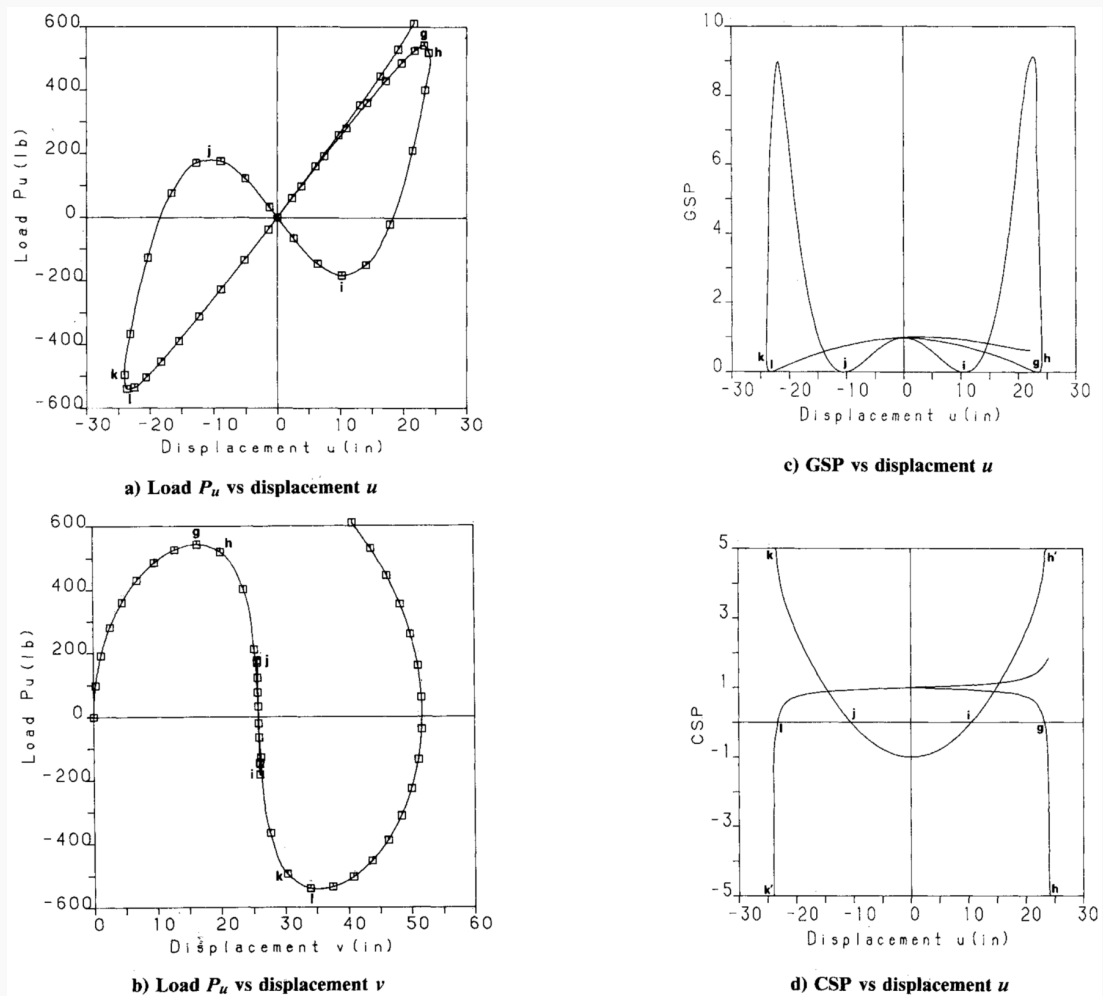
Para dois casos de carregamento (carga horizontal como imperfeição: $P_u = 0,05 P_v$; e carga vertical como imperfeição: $P_v = 0,05 P_u$), os resultados mostram curvas do tipo laço com quatro pontos limite e dois pontos de retrocesso. O GDC rastreia o caminho completo de equilíbrio com sucesso, invertendo automaticamente a direção de carregamento nos pontos limite. O GSP exibe comportamento suave ao longo de toda a trajetória, enquanto o CSP apresenta descontinuidades acentuadas nos pontos de retrocesso.

Figura 19: Resultados gerais do Exemplo 1 para o primeiro caso de carregamento.



Fonte: Adaptado de YANG, Y. -B.; SHIEH, M. -S. (1990)

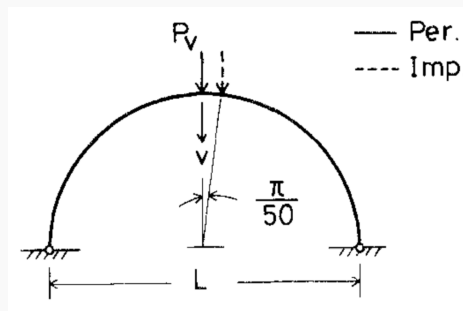
Figura 20: Resultados gerais do Exemplo 1 para o segundo caso de carregamento.



Fonte: Adaptado de YANG, Y. -B.; SHIEH, M. -S. (1990)

Exemplo 2 – Arco circular bi-articulado

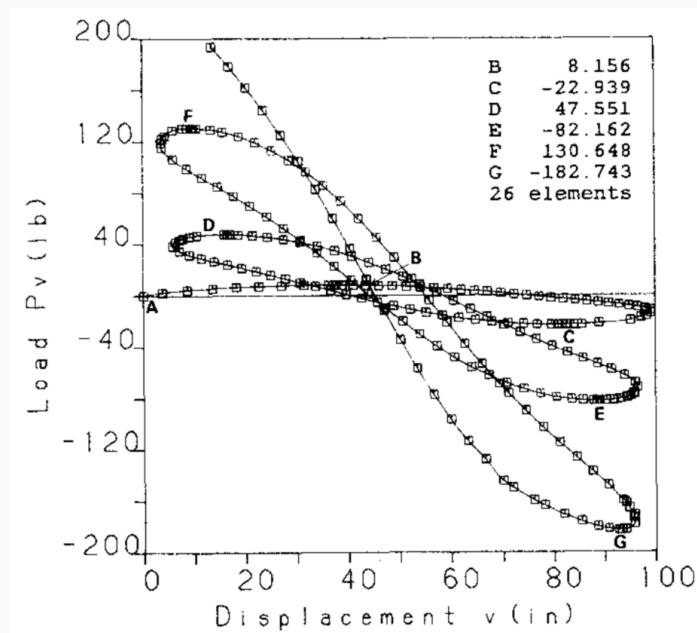
Figura 21: Treliza de dois membros.



Fonte: Adaptado de YANG, Y. -B.; SHIEH, M. -S. (1990)

Representado por 26 elementos de viga plana, o arco é submetido a carregamentos simétrico e assimétrico (com imperfeição de posição). Os gráficos de carga-deslocamento exibem respostas de pós-flambagem com múltiplas oscilações, demonstrando a capacidade do método de acompanhar trajetórias complexas com grandes deformações.

Figura 22: Resultados gerais do Exemplo 2.



Fonte: Adaptado de YANG, Y. -B.; SHIEH, M. -S. (1990)

Conclusões

O Método de Controle de Deslocamento Generalizado satisfaz simultaneamente os três critérios fundamentais para métodos de solução não-linear:

- (i) **estabilidade numérica** nas proximidades de pontos limite e de *snap-back*, mantendo λ_j^i e $\{u\}_j^i$ limitados;
- (ii) **ajuste automático do tamanho dos passos** por meio do GSP, que reflete diretamente a variação de rigidez da estrutura ao longo da trajetória;
- (iii) **capacidade autoadaptativa** de inversão da direção de carregamento, usando o sinal do GSP como indicador exclusivo dos pontos limite. O método pode ser implementado diretamente em programas de elementos finitos de propósito geral para análise não-linear geométrica.

APÊNDICE A - ROTINAS NUMÉRICAS IMPLEMENTADAS PARA SOLUÇÃO DOS TRABALHOS PRÁTICOS

A.1 - Início do documento, classes e métodos auxiliares e main

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.ticker as ticker
4  from pathlib import Path
5
6  # Cores estritamente escuras e distintas
7  _GLOBAL_COLORS = [
8      "#000080", # Azul Marinho
9      "#008000", # Verde Escuro
10     "#f58231", # Laranja Queimado
11     "#911eb4", # Roxo Escuro
12     "#00002D", # Preto Azulado
13     "#469990", # Cerceta / Azul Petroleo
14     "#e6194b", # Vermelho Carmesim
15     "#9a6324", # Marrom
16     "#808000", # Oliva
17     "#1976d2", # Azul Forte
18     "#388e3c", # Verde Floresta
19     "#7b1fa2", # Violeta Profundo
20     "#f57c00", # Laranja Escuro
21     "#5d4037", # Marrom Cafe
22     "#006064" # Ciano Escuro
23 ]
24 _global_color_idx = 0
25
26 # Marcadores para as curvas
27 _MARKERS = ["o", "s", "d", "+", "*"]
28
29 # Pasta de output
30 _ROOT = Path(__file__).parent
31 _DOC_DIR = _ROOT / "doc_latex"
32 _FIG_DIR = _DOC_DIR / "figuras"
33 OUT_DIR = _FIG_DIR / "resultados"
34
35 def _get_next_color():
36     """Retorna a proxima cor garantindo a exclusividade no ciclo."""
37     global _global_color_idx
38     color = _GLOBAL_COLORS[_global_color_idx % len(_GLOBAL_COLORS)]
39     _global_color_idx += 1
40     return color
41
42 def _comma_fmt(x, pos):
43     """Formata os rotulos dos eixos com virgula."""
44     return f"{x:g}".replace('.', ',')
45
46 def _apply_pi_ticks(ax, max_theta):
47     """Mapeia os eixos com fracoes notaveis de pi."""
48     if max_theta <= np.pi/2 + 0.1:
49         # Fracoes notaveis incluindo pi/12 e 5pi/12
50         ticks = [0, np.pi/12, np.pi/6, np.pi/4, np.pi/3, 5*np.pi/12, np.pi/2]

```

```

51     labels = ["0", r"$\pi/12$", r"$\pi/6$", r"$\pi/4$", r"$\pi/3$", r"$5\pi/12$", r"$\pi/2$"]
52     elif max_theta <= np.pi + 0.1:
53         ticks = [0, np.pi/4, np.pi/2, 3*np.pi/4, np.pi]
54         labels = ["0", r"$\pi/4$", r"$\pi/2$", r"$3\pi/4$", r"$\pi$"]
55     else:
56         ticks = [
57             0, np.pi/4, np.pi/2, 3*np.pi/4, np.pi,
58             5*np.pi/4, 3*np.pi/2, 7*np.pi/4, 2*np.pi
59         ]
60         labels = [
61             "0", r"$\pi/4$", r"$\pi/2$", r"$3\pi/4$", r"$\pi$",
62             r"$5\pi/4$", r"$3\pi/2$", r"$7\pi/4$", r"$2\pi$"
63         ]
64     ax.set_xticks(ticks)
65     ax.set_xticklabels(labels)
66
67
68     class _Helpers:
69         """Classe contendo ferramentas numericas comuns."""
70
71         @staticmethod
72         def solve_newton_raphson(res_func, hess_func, x0, tol=1e-3, max_it=50):
73             """Metodo de Newton-Raphson para equacoes nao lineares 1D."""
74             x = float(x0)
75             for i in range(max_it):
76                 res = res_func(x)
77                 if abs(res) <= tol:
78                     return x, i
79                 hess = hess_func(x)
80                 dx = -res / hess
81                 x = x + dx
82             raise ValueError("Newton-Raphson nao convergiu.")
83
84
85     class Plotter:
86         """Classe especifica para gerenciar todos os graficos solicitados."""
87
88         def __init__(self):
89             self._fig_list = []
90             self._fig_names = []
91             self._curr_ax = None
92             self._twin_ax = None
93             self._marker_idx = 0
94
95         def create_plot(self, tp_name, aspect_equal=False, fig = None):
96             """Inicializa uma nova figura com a cor de fundo solicitada."""
97             fig, ax = plt.subplots(figsize=(8, 6) if fig == None else fig)
98
99             # Aplicando a cor de fundo requerida para a figura e o eixo
100             fig.patch.set_facecolor('#f9f9f9')
101             ax.set_facecolor('#f9f9f9')
102
103             if aspect_equal:
104                 ax.set_aspect('equal')
105             self._curr_ax = ax
106             self._twin_ax = None
107             self._marker_idx = 0

```

```

108         self._fig_list.append(fig)
109         self._fig_names.append(tp_name)
110         return ax
111
112     def add_curve(
113         self,
114         x, y,
115         label=None,
116         sec_x=False,
117         linestyle="-", color=None, marker=None,
118         no_legend=False
119     ):
120         """Adiciona uma curva, permitindo sobreposicao de cor e marcador."""
121         if self._curr_ax is None:
122             return None, None
123
124         if color is None:
125             color = _get_next_color()
126         if marker is None:
127             marker = _MARKERS[self._marker_idx % len(_MARKERS)]
128             self._marker_idx += 1
129
130         mk_ev = max(1, int(len(x) / 12.0))
131
132         if sec_x:
133             if self._twin_ax is None:
134                 self._twin_ax = self._curr_ax.twinx()
135                 ax = self._twin_ax
136             else:
137                 ax = self._curr_ax
138
139         ax.plot(
140             x, y, color=color, marker=marker, markersize=4,
141             markevery=mk_ev, label=label if not no_legend else None,
142             linestyle=linestyle
143         )
144         return color, marker
145
146     def configure(self, x_lab, y_lab, title, sec_x_lab=None,
147                   log_y=False, is_theta_x=False, is_theta_sec_x=False,
148                   max_theta=np.pi/2, legend_loc='best', y_min=None):
149         """Configura rotulos, legendas unificadas, grid e formatacao."""
150         if self._curr_ax is None:
151             return
152
153         self._curr_ax.set_xlabel(x_lab)
154         self._curr_ax.set_ylabel(y_lab)
155         self._curr_ax.set_title(title)
156
157         if sec_x_lab and self._twin_ax:
158             self._twin_ax.set_xlabel(sec_x_lab)
159
160         if log_y:
161             self._curr_ax.set_yscale('symlog', linthresh=0.1)
162
163         if y_min is not None:
164             self._curr_ax.set_ylim(bottom=y_min)
165

```



```

166     self._curr_ax.minorticks_on()
167     self._curr_ax.grid(True, which='both', alpha=0.3)
168
169     for ax in [self._curr_ax, self._twin_ax]:
170         if ax is not None:
171             ax.xaxis.set_major_formatter(ticker.FuncFormatter(_comma_fmt))
172             ax.yaxis.set_major_formatter(ticker.FuncFormatter(_comma_fmt))
173
174         if is_theta_x:
175             _apply_pi_ticks(self._curr_ax, max_theta)
176         if is_theta_sec_x and self._twin_ax:
177             _apply_pi_ticks(self._twin_ax, max_theta)
178
179     ln_1, lab_1 = self._curr_ax.get_legend_handles_labels()
180     ln_2, lab_2 = [], []
181     if self._twin_ax is not None:
182         ln_2, lab_2 = self._twin_ax.get_legend_handles_labels()
183
184     lns = ln_1 + ln_2
185     labs = lab_1 + lab_2
186
187     if lns:
188         self._curr_ax.legend(lns, labs, loc=legend_loc)
189
190     def save_all_and_show(self):
191         """Salva todos os graficos em disco e os exibe no console/tela."""
192         name_counts = {}
193         for fig, name in zip(self._fig_list, self._fig_names):
194             if name not in name_counts:
195                 name_counts[name] = 1
196             else:
197                 name_counts[name] += 1
198
199         current_counts = {name: 0 for name in name_counts}
200
201         for fig, name in zip(self._fig_list, self._fig_names):
202             fig.tight_layout()
203             current_counts[name] += 1
204
205             if name_counts[name] > 1:
206                 filename = f"{name}-result{current_counts[name]}.pdf"
207             else:
208                 filename = f"{name}-result.pdf"
209
210             filepath = Path(OUT_DIR / filename).resolve()
211             # Garante a manutencao da cor de fundo ao salvar o PDF
212             fig.savefig(filepath, facecolor='#f9f9f9')
213             print(f"Salvo localmente: {filepath}")
214
215         plt.show()
216
217
218     class GeometricNonLinearAnalysis:
219         """Classe principal com metodos para cada TP da disciplina."""
220
221         def __init__(self):
222             self._plotter = Plotter()
223             self._helpers = _Helpers()

```

A.2 - Trabalho prático 01

```

1  def run_tp01(self):
2      k_mola = 1000.0
3      l_val = 100.0
4
5      theta = np.linspace(1e-5, np.pi/2, 200)
6      delta_v = l_val * (1 - np.cos(theta))
7      delta_h = l_val * np.sin(theta)
8      p_val = (k_mola / l_val) * (theta / np.sin(theta))
9
10     self._plotter.create_plot("TP01")
11     self._plotter.add_curve(delta_v, p_val, label=r"$\delta_v$")
12     self._plotter.add_curve(delta_h, p_val, label=r"$\delta_h$")
13     self._plotter.add_curve(theta, p_val, label=r"$\theta$", sec_x=True)
14     self._plotter.configure(
15         r"$\delta$ (cm)", r"$P$ (kN)",
16         r"TP01: $P \times \delta$ e $P \times \theta$",
17         sec_x_lab=r"$\theta$ (rad)",
18         is_theta_sec_x=True, max_theta=np.pi/2
19     )

```

A.3 - Trabalho prático 02

```

1  def run_tp02(self):
2      k_mola = 1.0
3      l_val = 100.0
4      theta = np.linspace(0, np.pi/2, 200)
5
6      delta_h = l_val * np.sin(theta)
7      delta_v = l_val * (1 - np.cos(theta))
8      p_val = k_mola * l_val * np.cos(theta)
9
10     self._plotter.create_plot("TP02")
11     self._plotter.add_curve(delta_v, p_val, label=r"$\delta_v$")
12     self._plotter.add_curve(delta_h, p_val, label=r"$\delta_h$")
13     self._plotter.add_curve(theta, p_val, label=r"$\theta$", sec_x=True)
14     self._plotter.configure(
15         r"$\delta$ (cm)", r"$P$ (kN)",
16         r"TP02: $P \times \delta$ e $P \times \theta$",
17         sec_x_lab=r"$\theta$ (rad)",
18         is_theta_sec_x=True, max_theta=np.pi/2
19     )

```

A.4 - Trabalho prático 03

```

1  def run_tp03(self):
2      k_mola = 1000.0
3      l_val = 100.0
4      theta = np.linspace(1e-5, np.pi/2 - 0.01, 200)

```

```

5
6     delta_h = l_val * (1 - np.cos(theta))
7     delta_v = l_val * np.sin(theta)
8     p_val = (4 * k_mola / l_val) * (theta / np.cos(theta))
9
10    self._plotter.create_plot("TP03")
11    self._plotter.add_curve(delta_v, p_val, label=r"$\delta_v$")
12    self._plotter.add_curve(delta_h, p_val, label=r"$\delta_h$")
13    self._plotter.add_curve(theta, p_val, label=r"$\theta$", sec_x=True)
14    self._plotter.configure(
15        r"$\delta$ (cm)", r"$P$ (kN)",
16        r"TP03: $P \times \delta$ e $P \times \theta$",
17        sec_x_lab=r"$\theta$ (rad)",
18        log_y=True,
19        is_theta_sec_x=True, max_theta=np.pi/2
20    )

```

A.5 - Trabalho prático 04

```

1  def run_tp04(self):
2      ei_val = 4e4
3      l_val = 500.0
4
5      theta = np.linspace(1e-5, 2 * np.pi, 200)
6      m_val = (ei_val * theta) / l_val
7      rho = l_val / theta
8
9      u_val = l_val - rho * np.sin(theta)
10     v_val = rho * (1 - np.cos(theta))
11
12     self._plotter.create_plot("TP04", fig=(4,6))
13     self._plotter.add_curve(u_val, m_val, label=r"$u$")
14     self._plotter.add_curve(v_val, m_val, label=r"$v$")
15     self._plotter.add_curve(theta, m_val, label=r"$\theta$", sec_x=True,
16                             linestyle="--")
17     self._plotter.configure(
18         r"Deslocamento (cm)", r"$M$ (N.cm)",
19         r"TP04: $M \times u$, $M \times v$ e $M \times \theta$",
20         sec_x_lab=r"$\theta$ (rad)",
21         is_theta_sec_x=True, max_theta=2*np.pi,
22         legend_loc='lower right'
23     )
24
25     self._plotter.create_plot("TP04", aspect_equal=True)
26     s = np.linspace(0, l_val, 11)
27
28     for t_val, t_label in zip([np.pi, 2 * np.pi], [r"\pi", r"2\pi"]):
29         rho_s = l_val / t_val
30         x_line = rho_s * np.sin(s / rho_s)
31         v_line = rho_s * (1 - np.cos(s / rho_s))
32         self._plotter.add_curve(
33             x_line, v_line, label=rf"$\theta = {t_label}$"
34         )
35
36     self._plotter.configure(

```

```

36         r"$x$ (cm)", r"$v$ (cm)", r"TP04: Linha Neutra",
37         legend_loc='lower right'
38     )

```

A.6 - Trabalho prático 05

```

1  def run_tp05(self):
2      ea_0 = 13865.0
3      l_val = 400.0
4      h_val = 20.0
5      y_val = np.arange(40.0, 19.0, -1.0)
6
7      beta_0 = np.arctan(h_val / l_val)
8      delta_v = h_val - y_val
9      beta = np.arctan(y_val / l_val)
10
11     f_val = 2 * ea_0 * np.tan(beta) * (np.cos(beta_0) - np.cos(beta))
12     n_val = f_val / (2 * np.sin(beta))
13
14     self._plotter.create_plot("TP05")
15     self._plotter.add_curve(delta_v, f_val, label=r"$F$")
16     self._plotter.add_curve(delta_v, n_val, label=r"$N$")
17     self._plotter.configure(
18         r"$\delta_v$ (cm)", r"Esforco (N)",
19         r"TP05: $F, N \times \delta_v$"
20     )

```

A.7 - Trabalho prático 06

```

1  def run_tp06(self):
2      lamb = np.linspace(0.1, 2.0, 200)
3
4      eps_alm = 0.5 * (1 - 1 / lamb**2)
5      eps_hyp = 1 - 1 / lamb
6      eps_log = np.log(lamb)
7      eps_biot = lamb - 1
8      eps_green = 0.5 * (lamb**2 - 1)
9
10     self._plotter.create_plot("TP06")
11     self._plotter.add_curve(lamb, eps_alm, label=r"Almansi")
12     self._plotter.add_curve(lamb, eps_hyp, label=r"Hiperbolica")
13     self._plotter.add_curve(lamb, eps_log, label=r"Logaritmica")
14     self._plotter.add_curve(lamb, eps_biot, label=r"Biot")
15     self._plotter.add_curve(lamb, eps_green, label=r"Green")
16     self._plotter.configure(
17         r"$\lambda$", r"$\epsilon^*$",
18         r"TP06: Medidas de Deformacao",
19         log_y=False, y_min=-10.0
20     )
21
22     self._plotter.create_plot("TP06")

```

```

23 self._plotter.add_curve(lamb, lamb**3, label=r"Almansi")
24 self._plotter.add_curve(lamb, lamb**2, label=r"Hiperbolica")
25 self._plotter.add_curve(lamb, lamb, label=r"Logaritmica")
26 self._plotter.add_curve(lamb, np.ones_like(lamb), label=r"Biot")
27 self._plotter.add_curve(lamb, 1 / lamb, label=r"Green")
28 self._plotter.configure(
29     r"$\lambda$", r"$\frac{\sigma^*}{\sigma_B}$",
30     r"TP06: Tensoes Normalizadas", legend_loc='upper center'
31 )

```

A.8 - Trabalho prático 08

```

1  def run_tp08(self):
2      ea_0 = 133.865
3      h_val = 2.0
4      l_val = 2.0
5
6      y_val = np.arange(2.0, -4.01, -0.02)
7      lamb = np.sqrt((l_val**2 + y_val**2) / (l_val**2 + h_val**2))
8      cos_alpha = y_val / np.sqrt(y_val**2 + l_val**2)
9
10     k_const = 3 * ea_0 * cos_alpha
11
12     p_green = -k_const * 0.5 * lamb * (lamb**2 - 1)
13     p_biot = -k_const * (lamb - 1)
14     p_log = -k_const * (np.log(lamb) / lamb)
15     p_hyp = -k_const * ((lamb - 1) / lamb**3)
16     p_alm = -k_const * ((lamb**2 - 1) / lamb**5)
17
18     idx1 = np.where(y_val >= -1e-5)[0]
19     idx2 = np.where((y_val <= 1e-5) & (y_val >= -2.0 - 1e-5))[0]
20     idx3 = np.where(y_val <= -2.0 + 1e-5)[0]
21
22     curves = [
23         (p_green, r"Green"),
24         (p_biot, r"Biot"),
25         (p_log, r"Logaritmica"),
26         (p_hyp, r"Hiperbolica"),
27         (p_alm, r"Almansi")
28     ]
29
30     self._plotter.create_plot("TP08")
31     for p_curve, label in curves:
32         # 1. Ramo inferior
33         c, m = self._plotter.add_curve(
34             lamb[idx2], p_curve[idx2], label=label, linestyle="--",
35         )
36         # 2. Ramo superior (tracejado e sem marcador)
37         self._plotter.add_curve(
38             lamb[idx1], p_curve[idx1], color=c, marker="None", linestyle="--",
39             no_legend=True
40         )
41         # 3. Ramo de tracao
42         self._plotter.add_curve(
43             lamb[idx3], p_curve[idx3], color=c, marker=m, linestyle="-",

```

```

no_legend=True
    )
43
44
45 self._plotter.configure(
46     r"$\lambda$", r"$P$ (kN)", r"TP08: $P \times \lambda$"
47 )

```

A.9 - Trabalho prático 09

```

1  def run_tp09(self):
2      e_val = 20000.0
3      l_val = 150.0
4      h_val = 10.0
5      a_0 = 5.0
6      p_ext = 20.0
7
8      c_0 = np.sqrt(h_val**2 + l_val**2)
9      k_nr = (e_val * a_0) / (c_0**3)
10
11     def _res_func(v):
12         return k_nr * (v**3 + 3 * h_val * v**2 + 2 * h_val**2 * v) - p_ext
13
14     def _hess_func(v):
15         return k_nr * (3 * v**2 + 6 * h_val * v + 2 * h_val**2)
16
17     v_0 = (p_ext * l_val / (2 * e_val * a_0)) * (1 + (l_val / h_val)**2)
18
19     try:
20         v_sol, iters = self._helpers.solve_newton_raphson(
21             _res_func, _hess_func, v_0, tol=0.001
22         )
23         print("TP09 - Metodo de Newton-Raphson:")
24         print(f" -> Convergiu em {iters} iteracoes.")
25
26         v_0_str = f"{v_0:.4f}".replace('.', ',')
27         v_sol_str = f"{v_sol:.4f}".replace('.', ',')
28         pct_str = f"{(v_sol / v_0) * 100:.2f}".replace('.', ',')
29
30         print(f" -> Chute inicial (linear): {v_0_str} cm")
31         print(f" -> Deslocamento vertical final v: {v_sol_str} cm")
32         print(f" -> v / v_linear: {pct_str} %")
33
34     except ValueError as e:
35         print(f"TP09 - Erro: {e}")

```